Name: Shivasachin S

REG No:192324276

LIST OF EXPERIMENTS

Sl. No	Experiment	CO
1.	Configuration of Network Devices using Packet Tracer tools (Hub, Switch, Ethernet, Broadcast).	**CO1**
2.	Design and Configuration of Star Topologies using Packet Tracer.	**CO1**
3.	Design and Configuration of BUS Topologies using Packet Tracer.	**CO1**
4.	Design and Configuration of RING Topologies using Packet Tracer.	**CO1**
5.	Design and Configuration of Mesh Topologies using Packet Tracer.	**CO1**
6.	Design and Configuration of Tree Topologies using Packet Tracer.	**CO1**
7.	Design and Configuration of Hybrid Topologies using Packet Tracer.	**CO1**
8.	Data Link Layer Traffic Simulation using Packet Tracer Analysis of ARP.	**CO2**
9.	Data Link Layer Traffic Simulation using Packet Tracer Analysis of CSMA/CD & CSMA/CA.	**CO2**
10.	Making Computer Lab in Cisco Packet Tracer.	**CO2**
11.	Designing two different network with Static Routing techniques using Packet Tracer.	**CO3**
12.	Design the Functionalities and Exploration of TCP using Packet Tracer.	**CO4**
13.	Design the network model for Subnetting – Class C Addressing using Packet Tracer.	**CO4**
14.	Simulating X, Y, Z Company Network Design and simulate using Packet Tracer.	**C05**
15.	Configuration of DHCP (dynamic host configuration protocol) in packet Tracer.	**CO4**
16.	Configuration of firewall in packet tracer.	**C05**
17.	Make a Computer Lab to transfer a message from one node to another to design and simulate using Cisco Packet Tracer.	**CO5**

18.	Simulate a Multimedia Network in Cisco Packet Tracer.	**CO4**
19.	IoT based smart home applications.	**CO3**
20.	Implementation of IoT based smart gardening.	**CO2**
21.	Implementation of IoT devices in networking.	**CO4**
22.	IoT based AAA Local and Server based authentication configuration.	**CO4**
23.	Transport layer protocol header analysis using Wire shark- TCP and UDP.	**CO5**
24.	Network layer protocol header analysis using Wire shark – SMTP and ICMP.	**CO4**
25.	Network layer protocol header analysis using Wire shark – ARP and HTTP.	**CO3**
26.	Implementation of date and time display from client to server using TCP sockets in java/C.	**CO2**
27.	Implementation of a DNS server and client in java/C using UDP sockets.	**CO3**
28.	Developing a client that contacts a given DNS server to resolve a given hostname in java/C.	**CO4**
29.	Creating the applications using TCP echo server and client in java/C.	**CO4**
30.	Creating the applications using TCP chat client and chat server in java/C.	**CO3**
31.	Implementing ARP protocols in java/C.	**CO5**
32.	Implementation of Bit stuffing mechanism using C.	**CO5**
33.	Implementing the applications using TCP file transfer in java/C.	**CO5**
34.	Implementing the simulation of error correction code - CRC in java/C.	**CO5**
35.	Implementing the sliding window protocol in java/C.	**CO3**

Date:

EXPERIMENT-1

CONFIGURATION OF NETWORK COMPONENTS

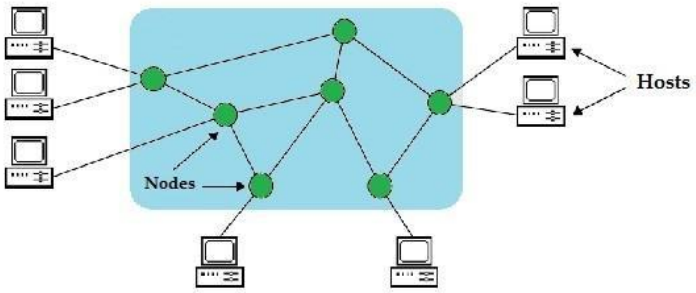
Aim: To Study the following Network Devices in Detail

- PC
- Server
- Repeater
- Hub

- Switch
- Bridge
- Router
- Gate Way
- Transmission medium

Apparatus (Software): CISCO Packet tracer.

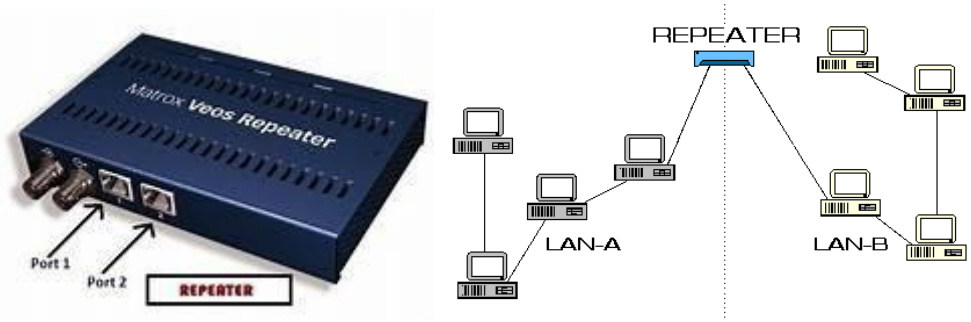
1. Node: In a communications **network**, a **network node** is a connection point that can receive, create, store or send data along distributed **network** routes.



Computer Network

2. **Repeater:** Functioning at Physical Layer.

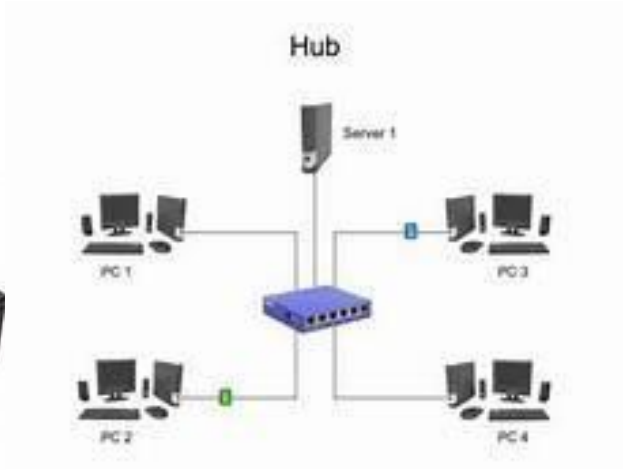
A **repeater** is an electronic device that receives a signal and retransmits it at a higher level and/or higher power, or onto the other side of an obstruction, so that the signal can cover longer distances.



3. Hub: Ethernet hub, **active hub**, **network hub**, **repeater hub**

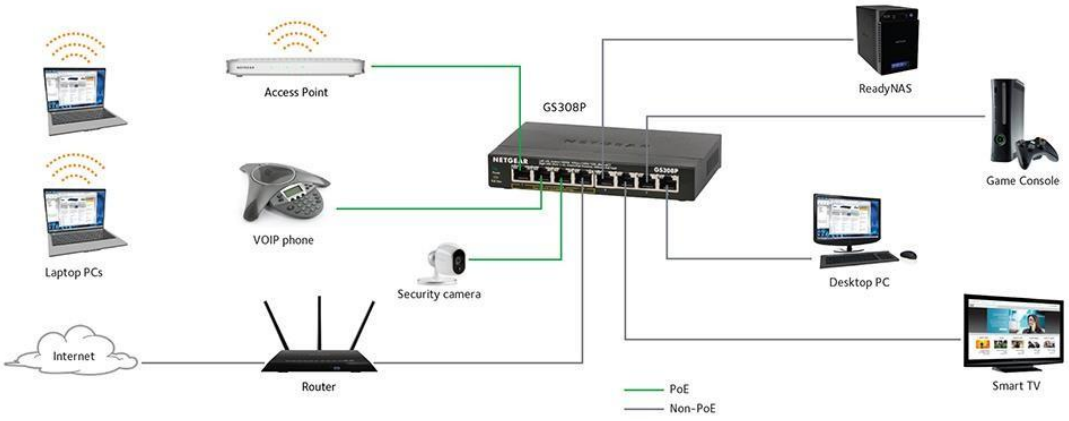
Hub or concentrator is a device for connecting multiple twisted pair or fiber optic Ethernet devices together and making them act as a single network segment. Hubs work at the physical layer (layer 1) of the OSI model. The device is a form of multiport repeater. Repeater hubs also participate in collision detection, forwarding a jam signal to all ports if it detects a collision.





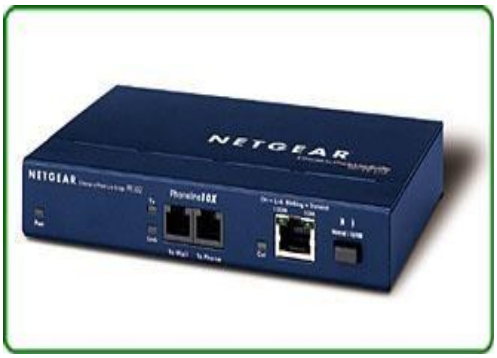
4. **Switch:** A **network switch** or **switching hub** is a computer networking device that connects network segments. The term commonly refers to a network bridge that processes and routes data at the data link layer (layer 2) of the OSI model. Switches that additionally process data at the network layer (layer 3 and above) are often referred to as Layer 3 switches or multilayer switches.

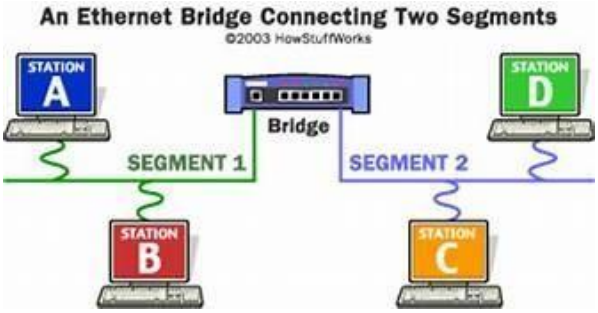




5. **Bridge:** A **network bridge** connects multiple network segments at the data link layer (Layer 2) of the OSI model. In Ethernet networks, the term bridge formally means a device that behaves according to the IEEE 802.1D standard. A bridge and switch are very much alike; a switch being a bridge with numerous ports. Switch or Layer 2 switch is often used interchangeably with bridge.

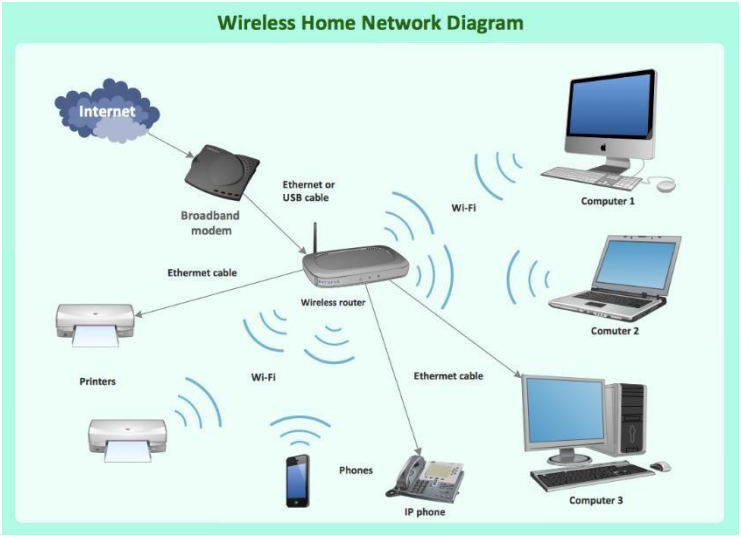
Bridges can analyze incoming data packets to determine if the bridge is able to send the given packet to another segment of the network.





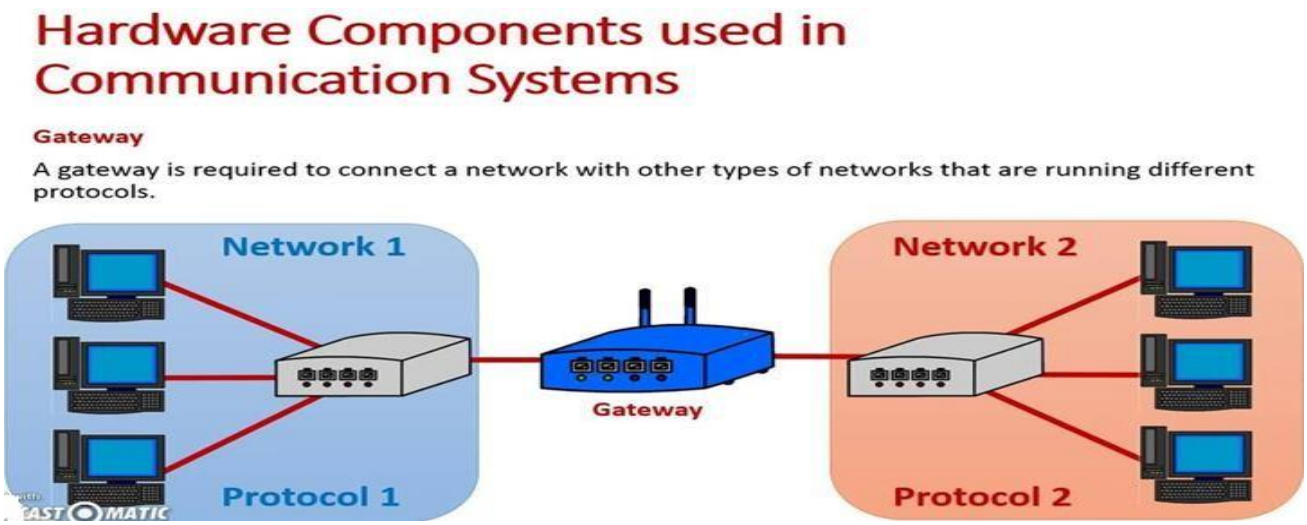
6. **Router:** A **router** is an electronic device that interconnects two or more computer networks, and selectively interchanges packets of data between them. Each data packet contains address information that a router can use to determine if the source and destination are on the same network, or if the data packet must be transferred from one network to another. The multiple routers are used in a large collection of interconnected networks, the routers exchange information about target system addresses, so that each router can build up a table showing the preferred paths between any two systems on the interconnected networks.



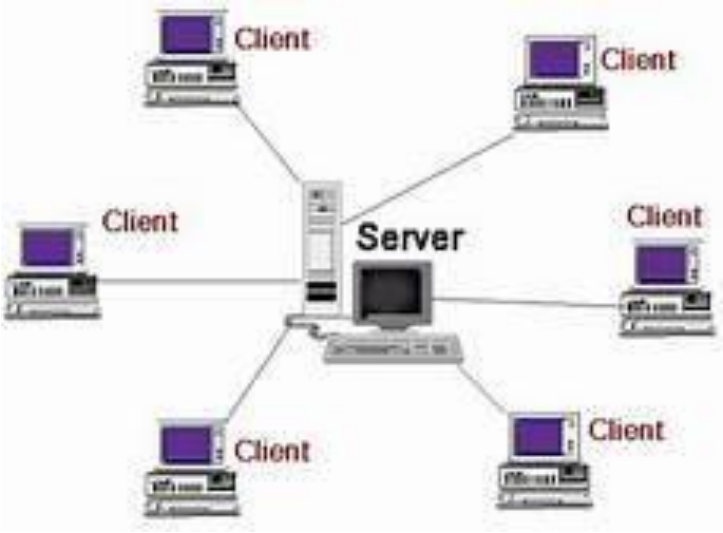


7. **Gate Way:** In a communication network, a network node equipped for interfacing with another network that uses different protocols. A gateway may contain devices such as protocol translators, impedance matching devices, rate converters, fault isolators, or signal translators as necessary to provide system interoperability. It also requires the establishment of mutually acceptable administrative procedures between both networks.
   - A protocol translation/mapping gateway interconnects networks with different network protocol technologies by performing the required protocol conversions.

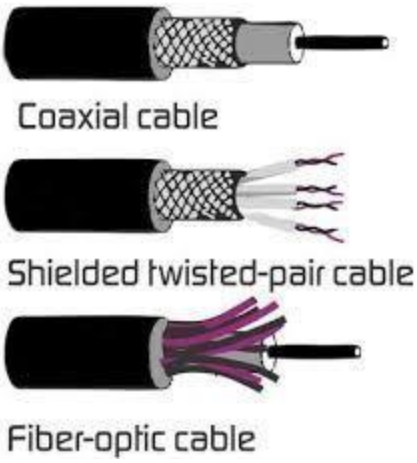


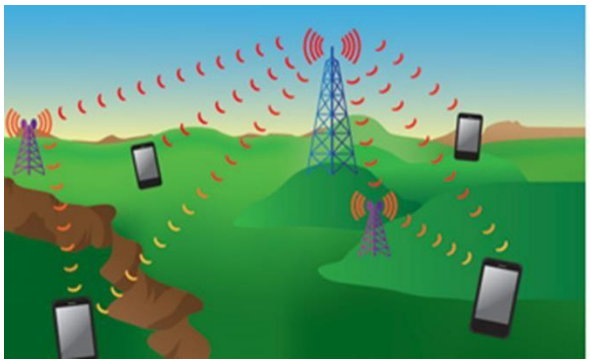


8. Server: A server is a type of computer or device on a network that manages network resources. Servers are often dedicated, meaning that they perform no other tasks besides their server tasks. On multiprocessing operating systems, however, a single computer can execute several programs at once. A server in this case could refer to the program that is managing resources rather than the entire computer.



9. **Transmission media**: The medium through which the signals travel from one device to another. These are classified as guided and unguided. Guided media are those that provide a conduit from one device to another. Eg. Twisted pair, coaxial cable etc. Unguided media transport signals without using physical cables. Eg. Air.





Result: Thus the network components are studied in detail.

Date:

EXPERIMENT-2

IMPLEMENTATION OF STAR TOPOLOGY USING PACKET TRACER

Aim: To Implement a star topology using packet tracer and hence to transmit data between the devices connected using star topology.

Software/Apparatus required: Packet Tracer/End devices, bridge, connectors.

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Topology – Connecting the Hosts to Switches

Select a switch, by clicking once on **Switches** and once on a **2950-24** switch.

Add the switch by moving the plus sign "+"

Step 5: Connect PCs to switch by first choosing Connections

Click once on the **Copper Straight-through** cable

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Switch0**

Click once on **Switch0**

Notice the green link lights on **PC** Ethernet NIC and amber light **Switch port**. The switch port is temporarily not forwarding frames, while it goes through the stages for the Spanning Tree Protocol (STP) process. After about 30 seconds the amber light will change to green indicating that the port has entered the forwarding stage. Frames can now forwarded out the switch port.

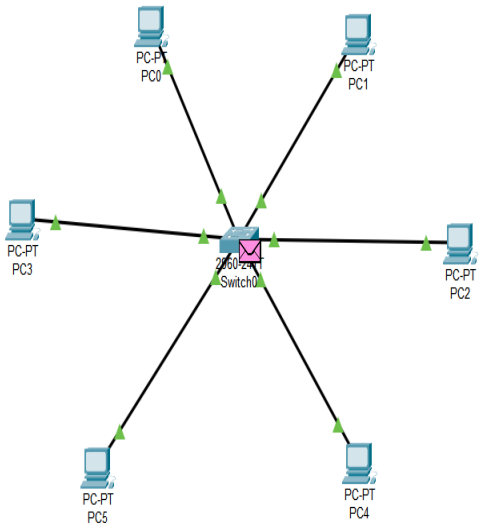
Step 6: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be Configured on the devices. Click once on PC0. Choose the Config tab and click on FastEthernet0. Type the IP address in its field. Click on the subnet mask it will be

generated automatically.

Step 7: To confirm Data transfer between the devices

Click on the node. Select desktop option and then command prompt. Once the window pops up, ping the IP address of the device to which node0 is connected. Ping statistics will be displayed.





Result: Thus the Star topology is implemented with Packet Tracer simulation Tool.

Date:

EXPERIMENT-3

IMPLEMENTATION OF BUS TOPOLOGY USING PACKET TRACER

Aim: To Implement a Bus topology using packet tracer and hence to transmit data between the devices connected using Bus topology.

Software / Apparatus required: Packet Tracer / End devices, Hubs, connectors.

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Topology – Connecting the Hosts to Switches

Select a switch, by clicking once on **Switches** and once on a **2950-24** switch.

Add the switch by moving the plus sign "+"

Step 5: Connect PCs to switch by first choosing connections

Click once on the **Copper Straight-through** cable

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Switch0**

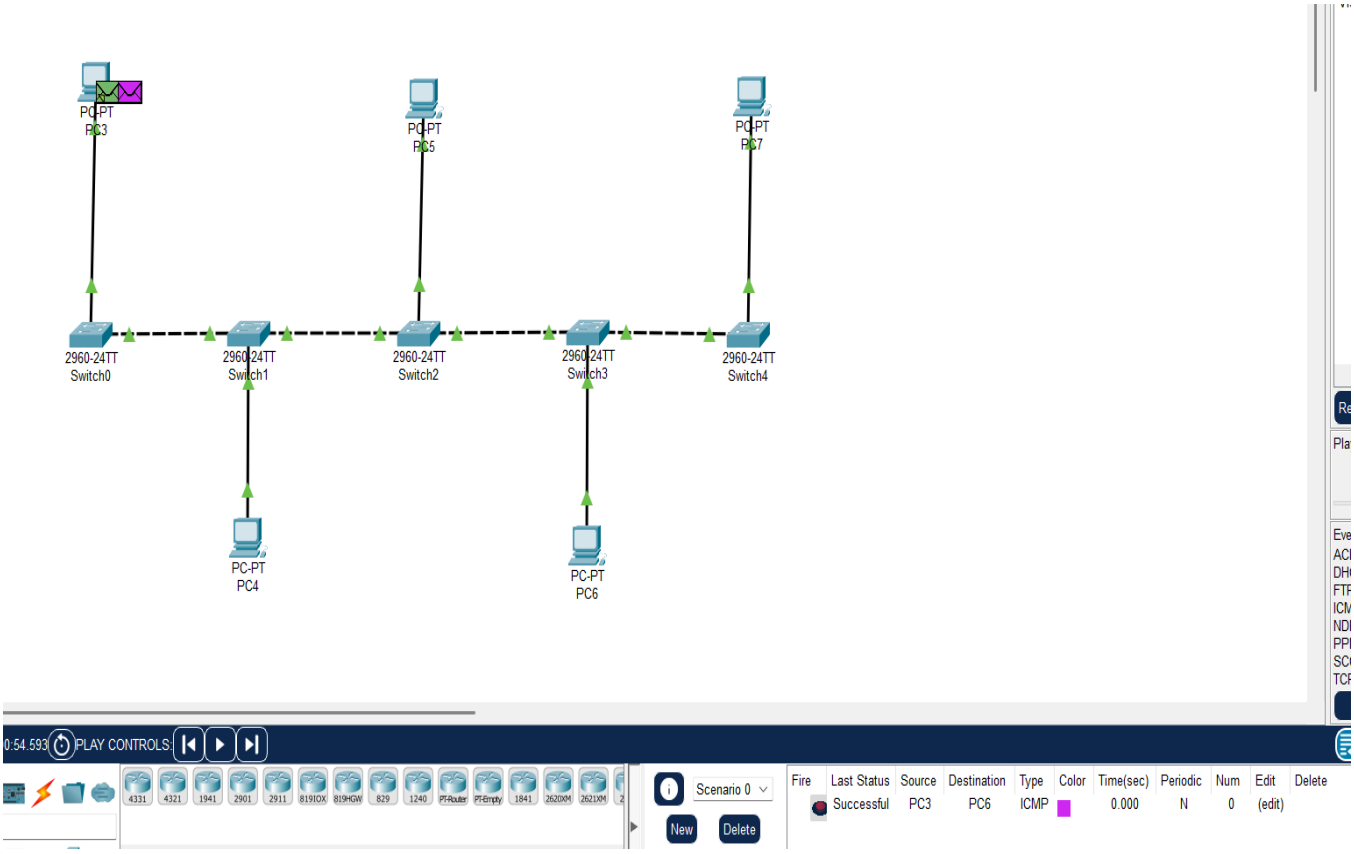
Click once on **Switch0**

Notice the green link lights on **PC** Ethernet NIC and amber light **Switch port**. The switch port is temporarily not forwarding frames, while it goes through the stages for the Spanning Tree Protocol (STP) process. After about 30 seconds the amber light will change to green indicating that the port has entered the forwarding stage. Frames can now forward out the switch port.

Step 6: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be configured on the devices. Click once on PC0. Choose the Config tab and click on automatically.

Click on the node. Select desktop option and then command prompt. Once the window pops up, ping the IP address of the device to which node0 is connected. Ping statistics will be displayed.



Result: Thus the Bus topology is implemented with Packet Tracer simulation Tool.

Date:

EXPERIMENT-4

IMPLEMENTATION OF RING TOPOLOGY USING PACKET TRACER

Aim: To Implement a Ring topology using packet tracer and hence to transmit data between the devices connected using Ring topology.

Software / Apparatus required: Packet Tracer / End devices, Hubs, Connectors.

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Topology – Connecting the Hosts to Switches

Select a switch, by clicking once on **Switches** and once on a **2950-24** switch.

Add the switch by moving the plus sign "+"

Step 5: Connect PCs to switch by first choosing connections

Click once on the **Copper Straight-through** cable

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Switch0**

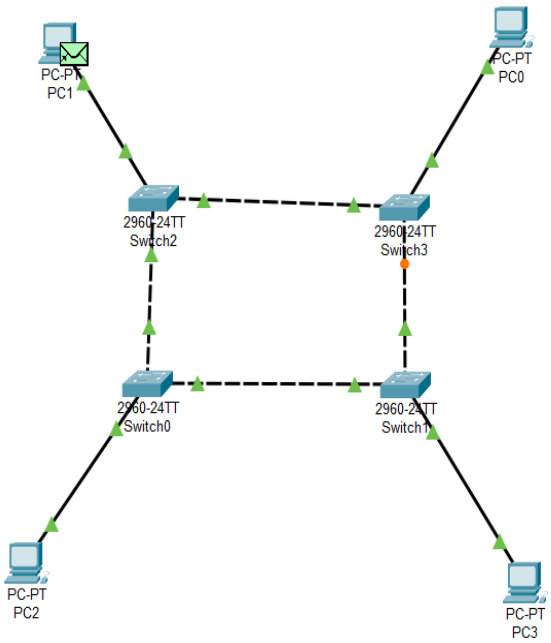
Click once on **Switch0**

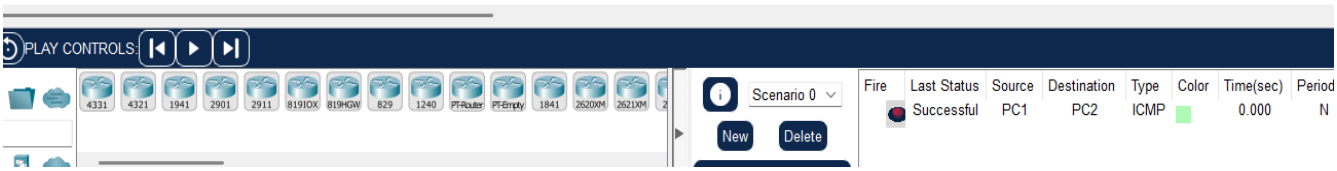
Notice the green link lights on **PC** Ethernet NIC and amber light **Switch port**. The switch port is temporarily not forwarding frames, while it goes through the stages for the Spanning Tree Protocol (STP) process. After about 30 seconds the amber light will change to green indicating that the port has entered the forwarding stage. Frames can now forward out the switch port.

Step 6: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be configured on the devices. Click once on PC0. Choose the Config tab and click on automatically.

Click on the node. Select desktop option and then command prompt. Once the window pops up, ping the IP address of the device to which node0 is connected. Ping statistics will be displayed.





Result: Thus the Ring topology is implemented with Packet Tracer simulation Tool.

Date:

EXPERIMENT-5

IMPLEMENTATION OF MESH TOPOLOGY USING PACKET TRACER

Aim: To Implement a Mesh topology using packet tracer and hence to transmit data between the devices connected using Mesh topology.

Software / Apparatus required: Packet Tracer / End devices, Hubs, Connectors.

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Topology – Connecting the Hosts to Switches

Select a switch, by clicking once on **Switches** and once on a **2950-24** switch.

Add the switch by moving the plus sign "+"

Step 5: Connect PCs to switch by first choosing connections

Click once on the **Copper Straight-through** cable

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Switch0**

Click once on **Switch0**

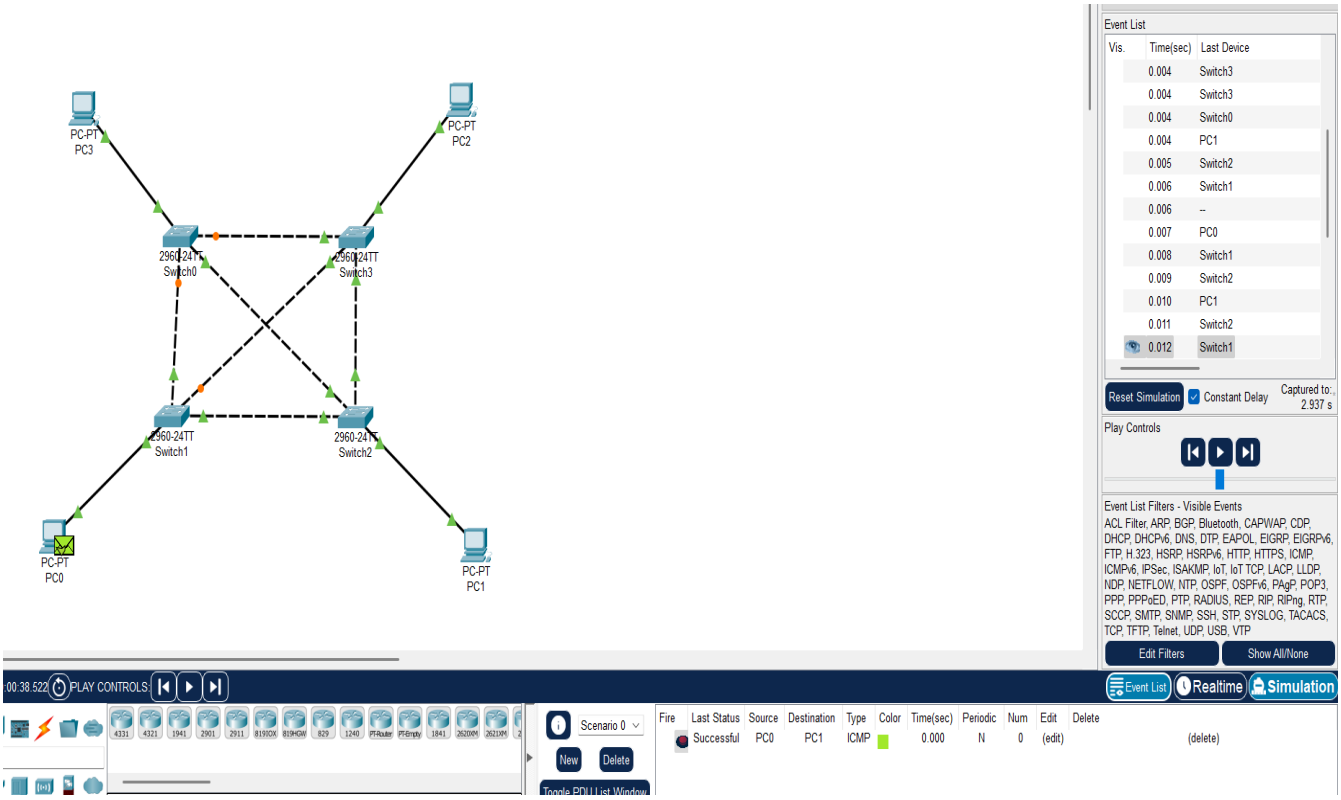
Notice the green link lights on **PC** Ethernet NIC and amber light **Switch port**. The switch port is temporarily not forwarding frames, while it goes through the stages for the Spanning Tree Protocol (STP) process. After about 30 seconds the amber light will change to green indicating that the port has entered the forwarding stage. Frames can now forward out the switch port.

Step 6: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be configured on the devices. Click once on PC0. Choose the Config tab and click on automatically.

Step 7: To confirm Data transfer between the devices

Click on the node. Select desktop option and then command prompt. Once the window pops up, ping the IP address of the device to which node0 is connected. Ping statistic will be displayed.



Result: Thus the Mesh topology is implemented with Packet Tracer simulation Tool.

Date:
Date:

DATE:

EXPERIMENT-6

IMPLEMENTATION OF TREE TOPOLOGY USING PACKET TRACER

Aim: To Implement a tree topology using packet tracer and hence to transmit data between the devices connected using tree topology.

Software / Apparatus required: Packet Tracer / End devices, Hubs, connectors.

Procedure:

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Star Topology – Connecting the Hosts to Hubs

Select a Hub, by clicking once on **Hub** and once on a **generic Hub**

Add the Hub by moving the plus sign "+"

Step 5: Connect PCs to Hub by first choosing Connections

Click once on the **Automatic cable selector**

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Hub0**

Click once on **Hub0**

Proceeding in this way create three star topologies

Step 6: Building the Tree Topology – Connecting the Hubs to Active Hub

Connect the hubs of star topologies to active hub to create tree topology.

Step 7: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be configured on the devices. Click once on PC0. Choose the Config tab and click on Fast Ethernet0. Type the IP address in its field. Click on the subnet mask. It will be generated automatically.

Step 8: Verifying Connectivity in Real time Mode Be sure you are in **Real time** mode.

Select the **Add Simple PDU** tool used to ping devices.
Click once on PC0, then once on PC3.
The PDU **Last Status** should show as **Successful**.

Step 9: **Verifying Connectivity in Simulation Mode**

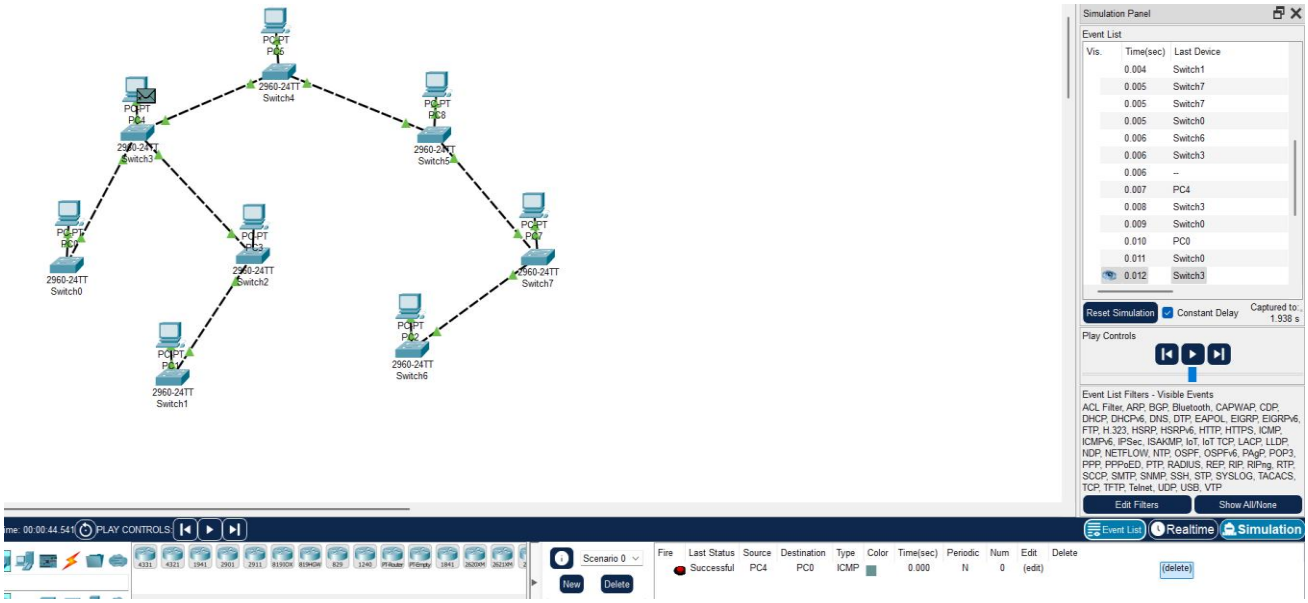
Be sure you are in **Simulation** mode.

Deselect all filters (All/None) and select only **ICMP**.

Select the **Add Simple PDU** tool used to ping devices

Click once on PC0, then once on PC3.

Continue clicking **Capture/Forward** button until the ICMP ping is completed. You should see the ICMP messages move between the hosts, hub and switch. The PDU **last status** should show as **Successful**.



Result: Thus the Tree topology is implemented with Packet Tracer simulation Tool.

Date:

DATE:

EXPERIMENT-7 IMPLEMENTATION OF HYBRID TOPOLOGY (BUS AND RING TOPOLOGY) USING PACKET TRACER

Aim: To Implement a hybrid topology using packet tracer and hence to transmit data between the devices connected using tree topology.

Software / Apparatus required: Packet Tracer / End devices, Hubs, connectors.

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Building the Topology – Adding Hosts

Single click on the **End Devices**.

Single click on the **Generic** host.

Move the cursor into topology area.

Single click in the topology area and it copies the device.

Step 4: Building the Bus Topology – Connecting the Hosts to Hubs

Select a Hub, by clicking once on **Hub** and once on a **generic Hub**

Add the Hub by moving the plus sign "+"

Step 5: Building the Ring Topology – Connecting the Hosts to Hubs

Select a Hub, by clicking once on **Hub** and once on a **generic Hub**

Add the Hub by moving the plus sign "+"

Step 5: Connect PCs to Hub by first choosing Connections

Click once on the **Automatic cable selector**

Click once on **PC2**

Choose **Fast Ethernet**

Drag the cursor to **Hub0**

Click once on **Hub0**

Proceeding in this way create three Bus topologies

Step 6: Building the Tree Topology – Connecting the Hubs to Active Hub

Connect the hubs of star topologies to active hub to create tree topology.

Step 7: Configuring IP Addresses and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses and Subnet Masks had to be configured on the devices. Click once on PC0. Choose the Config tab and click on FastEthernet0. Type the IP address in its field. Click on the subnet mask. It will be Generated automatically.

Step 8: Verifying Connectivity in Realtime Mode

Be sure you are in **Realtime** mode.

Select the **Add Simple PDU** tool used to ping devices.

Click once on PC0, then once on PC3.

The PDU **Last Status** should show as **Successful**.

Step 9: **Verifying Connectivity in Simulation Mode**

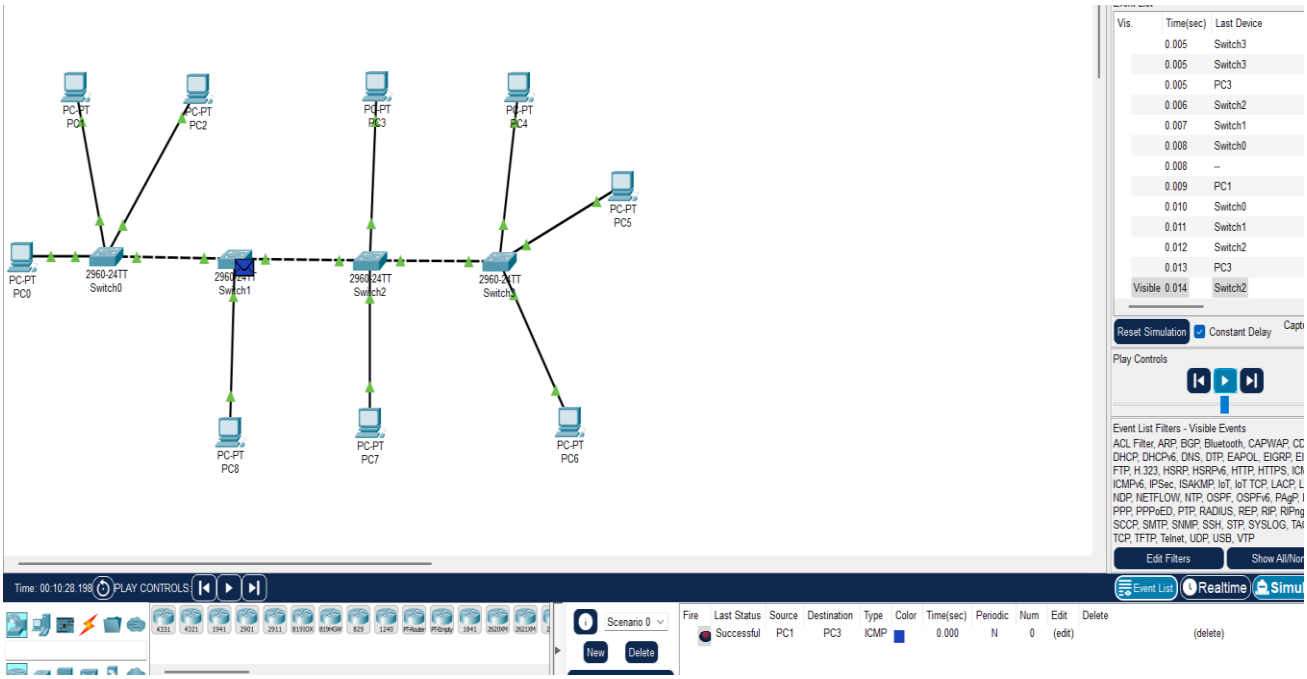
Be sure you are in **Simulation** mode.

Deselect all filters (All/None) and select only **ICMP**.

Select the **Add Simple PDU** tool used to ping devices

Click once on PC0, then once on PC3.

Continue clicking **Capture/Forward** button until the ICMP ping is completed. The ICMP messages move between the hosts, hub and switch. The PDU **Last Status** should show as **Successful**.



Result: Thus the Hybrid topology is implemented with Packet Tracer simulation Tool.

DATE:

EXPERIMENT-8 DATA LINK LAYER TRAFFIC SIMULATION USING PACKET TRACER ANALYSIS OF ARP

Aim: To implement Data Link Layer Traffic Simulation using Packet Tracer Analysis of ARP.

Software / Apparatus required: Packet Tracer / End devices, Switches, connectors.

Requirements:

1. End device - They are the devices through which we can pass message from one device to another and they are interconnected.

2.Switch/Hub - Interface Between two devices.

3.Cable - Used to connect two devices

Procedure:

1. Open packet tracer.
2. Click on the list the available capture interface.
3. Choose the PCS, server and Hub.
4. Later give connection from hub to the remaining pcs.
5. Give IP address to the pcs with configuration.
6. Simulate the source and destination.



Result: Thus the Data Link Layer Traffic Simulation using Packet Tracer Analysis of ARP is implemented.

DATE:

EXPERIMENT-9

DATA LINK LAYER TRAFFIC SIMULATION USING PACKET TRACER

ANALYSIS OF CSMA/CD & CSMA/CA

Aim: To implement Data Link Layer Traffic Simulation using Packet Tracer Analysis of CSMA/CD & CSMA/CA.

Software / Apparatus required: Packet Tracer / End devices, Switches, connectors.

Requirements:

1. End device - They are the devices through which we can pass message from one device to another and they are interconnected.

2.Switch/Hub - Interface Between two devices.

3.Cable - Used to connect two devices **Procedure:**

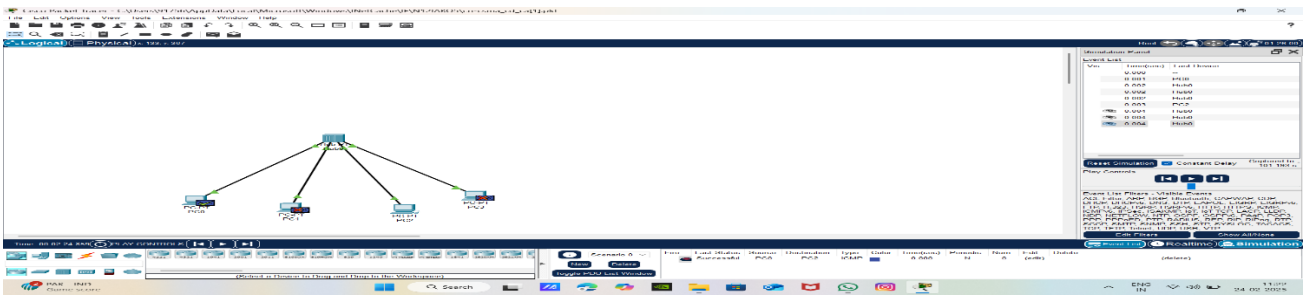
STEP 1: Click on end devices, select generic Pc's drag and drop it on the window.

Click on SWITCH drag and drop it on the window.

STEP 2: Select the straight through cable and connect all end device to switch. Assign the IP address for all end devices. (Double click the end device Select → desktop → IP configuration static)

STEP 3: Now set the IP address to Host A (192.168.1.1) in static mode. Similarly set IP address for Host B (192.168.1.2) and Host C (192.168.1.3)

STEP 4: To view the IP address, give ip config command in command prompt. Using ping command, we can establish communication between two host devices. STEP 5: Now display the packet transmission in simulation mode.



Result: Thus Data Link Layer Traffic Simulation using Packet Tracer Analysis of CSMA/CD & CSMA/CA is implemented successfully.

EXPERIMENT-10

MAKING COMPUTER LAB IN CISCO PACKET TRACER

Aim: Making Computer Lab in Cisco Packet Tracer.

Software / Apparatus required: Packet Tracer / End devices, Switches, connectors.

Procedure:

Step 1: Launch Cisco Packet Tracer and create a new project.

Step 2: Select the appropriate network devices for your lab. In this case, you will need computers, switches, and routers. You can find these devices in the "End Devices," "Switches," and "Routers" sections of the device list.

Step 3: Drag and drop a switch onto the workspace area. Connect the switch to the power source by clicking on the "Connection" option and selecting "Power."

Step 4: Connect computers to the switch by dragging and dropping them onto the workspace area. Click on the "Connection" option and select "Fast Ethernet" to connect the computers to the switch.

Step 5: Repeat Step 4 to add more computers to the lab. You can adjust the number of computers as per your requirements.

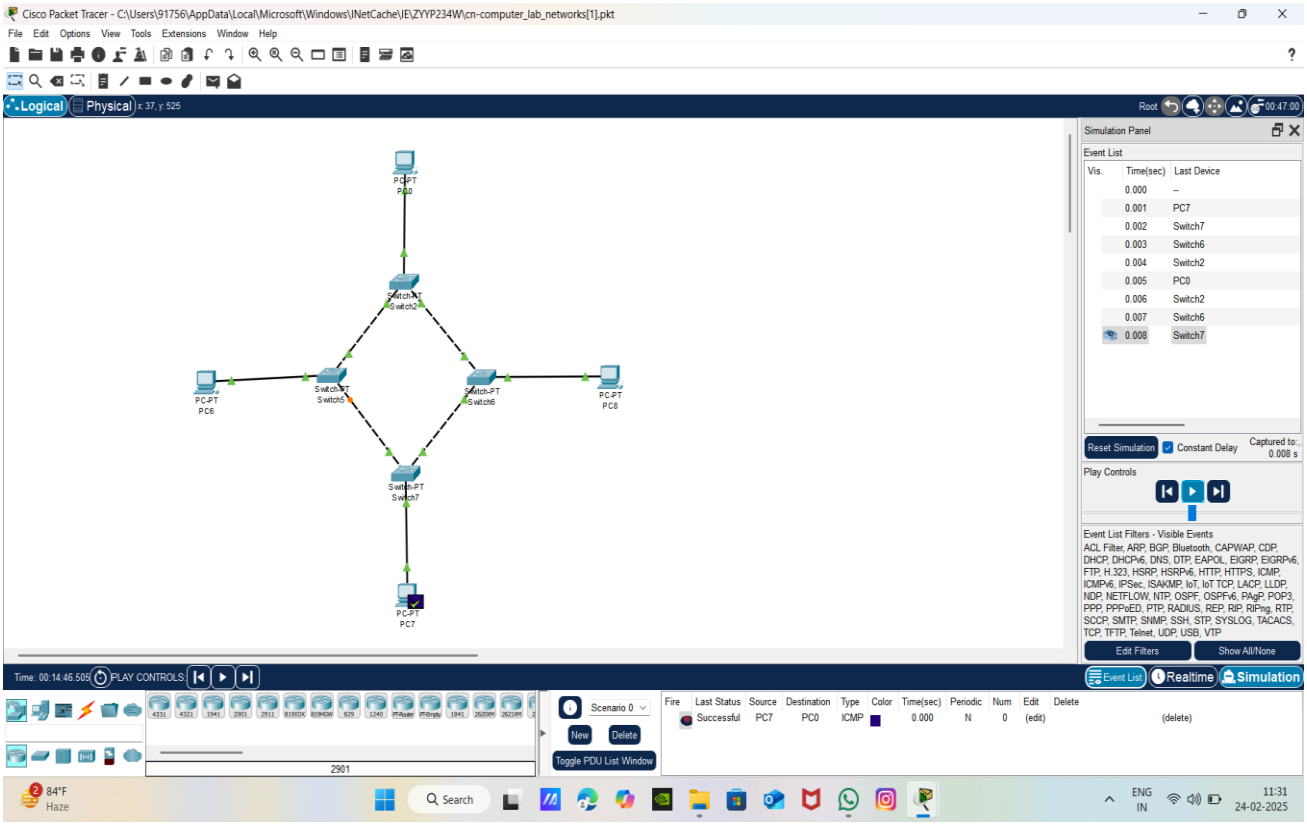
Step 6: Connect the switch to a router. Drag and drop a router onto the workspace area and connect it to the switch using a serial cable. To do this, click on the "Connection" option, select "Serial," and then select the appropriate serial interface on the router.

Step 7: Configure IP addresses on the computers. Select a computer, click on the "Desktop" tab in the device configuration panel, and configure the IP address, subnet mask, and default gateway for each computer.

Step 8: Configure IP addresses on the router interfaces. Select the router, click on the "CLI" tab in the device configuration panel, and enter the interface configuration mode. Assign IP addresses to the router interfaces connected to the switch and computers.

Step 9: Test connectivity. Open the command prompt on each computer and try to ping other computers and the router's interfaces to ensure connectivity.

Step 10: Customize and expand the lab as desired. You can add additional devices, configure VLANs, implement security measures, or set up servers within the lab environment.



Result: Thus the Computer Lab in Cisco Packet Tracer is set up successfully.

Date:

EXPERIMENT-11

CONFIGURATION OF A SIMPLE STATIC ROUTING IN PACKET TRACER USING A SIMPLE TOPOLOGY WITH TWO ROUTERS

Aim: To Configure a router using packet tracer software and hence to transmit data between the devices in real time mode and simulation mode.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Steps for building topology:

Step 1: Start Packet Tracer

Step 2: Choosing Devices and Connections

Step 3: Single click on the **End Devices**.

Single click on the **Generic** Host.

Place PC0, PC1 on topology area.

Connect PCs to Switch 1.

Similarly Place PC2, PC3 on topology area for receiver side

Connect these PCs with switch 1 and 2 respectively through connecting wires.

Select Router and place the router between two switches.

Connect these switches into router through connecting wires.

Step 3: Configuring IP Addresses, Gate Way and Subnet Masks on the Hosts

To start communication between the hosts IP Addresses, subnet Masks and Gate way had to be configured on the devices. Click once on PCs. Choose the Config tab and click on FastEthernet0. Type the IP address in its field. Based on router create gate way click on the subnet mask. It will be generated automatically. **Step 4: Verifying Connectivity in Real time Mode** Be sure you are in **Real time** mode.

Select the **Add Simple PDU** tool used to ping devices.

Click once on PC0, then once on PC3.

The PDU **Last Status** should show as **Successful**.

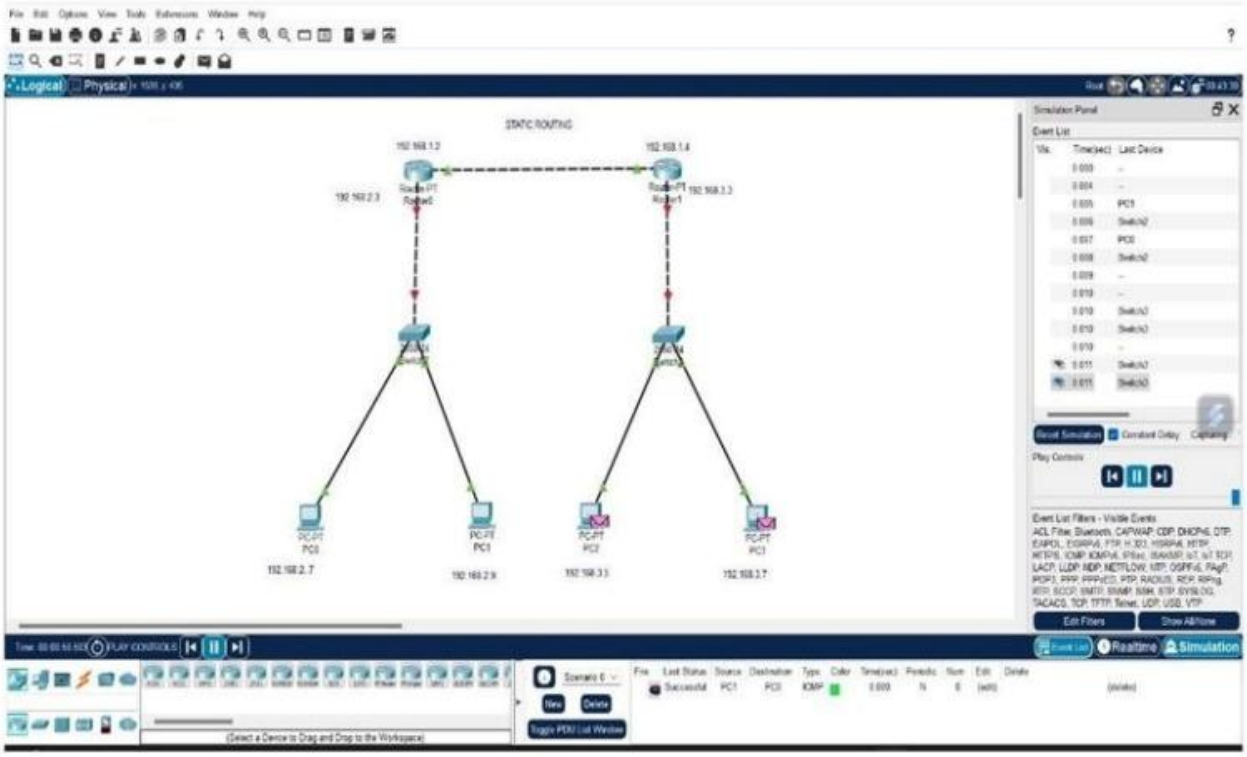
Step 5: **Verifying Connectivity in Simulation Mode**

Be sure you are in **Simulation** mode.

Deselect all filters (All/None) and select only **ICMP**.

Select the **Add Simple PDU** tool used to ping devices Click once on PC0, then once on PC3.

Continue clicking **Capture/Forward** button until the ICMP ping is completed. The ICMP messages move between the hosts, hub and switch. The PDU **Last Status** should show as **Successful**.



Result: Thus Configuration of a simple static routing in packet tracer using a simple topology with two routers was done successfully.

Date:

EXPERIMENT-12 DESIGN THE FUNCTIONALITIES AND EXPLORATION OF TCP USING PACKET TRACER

Aim: To design the Functionalities and Exploration of TCP using Packet Tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Step 1: Setup the network topology

To begin, we will create a simple network topology consisting of two computers connected by a router. Open Packet Tracer and drag two PCs and a router onto the workspace. Connect the two PCs to the router using Ethernet cables.

Step 2: Configure IP addresses

Next, we will configure IP addresses for the computers. Double-click on each PC to open the configuration window and navigate to the Desktop tab. Click on the IP Configuration icon and enter the IP address and subnet mask for each computer. For example, PC1 can have an IP address of 192.168.1.1 with a subnet mask of 255.255.255.0 and PC2 can have an IP address of 192.168.1.2 with the same subnet mask.

Step 3: Configure the router

Now, we will configure the router. Double-click on the router to open the configuration window and navigate to the CLI tab.

COMMANDS:

enable configure terminal

interface FastEthernet0/0

ip address 192.168.1.254 255.255.255.0 no

shutdown

exit

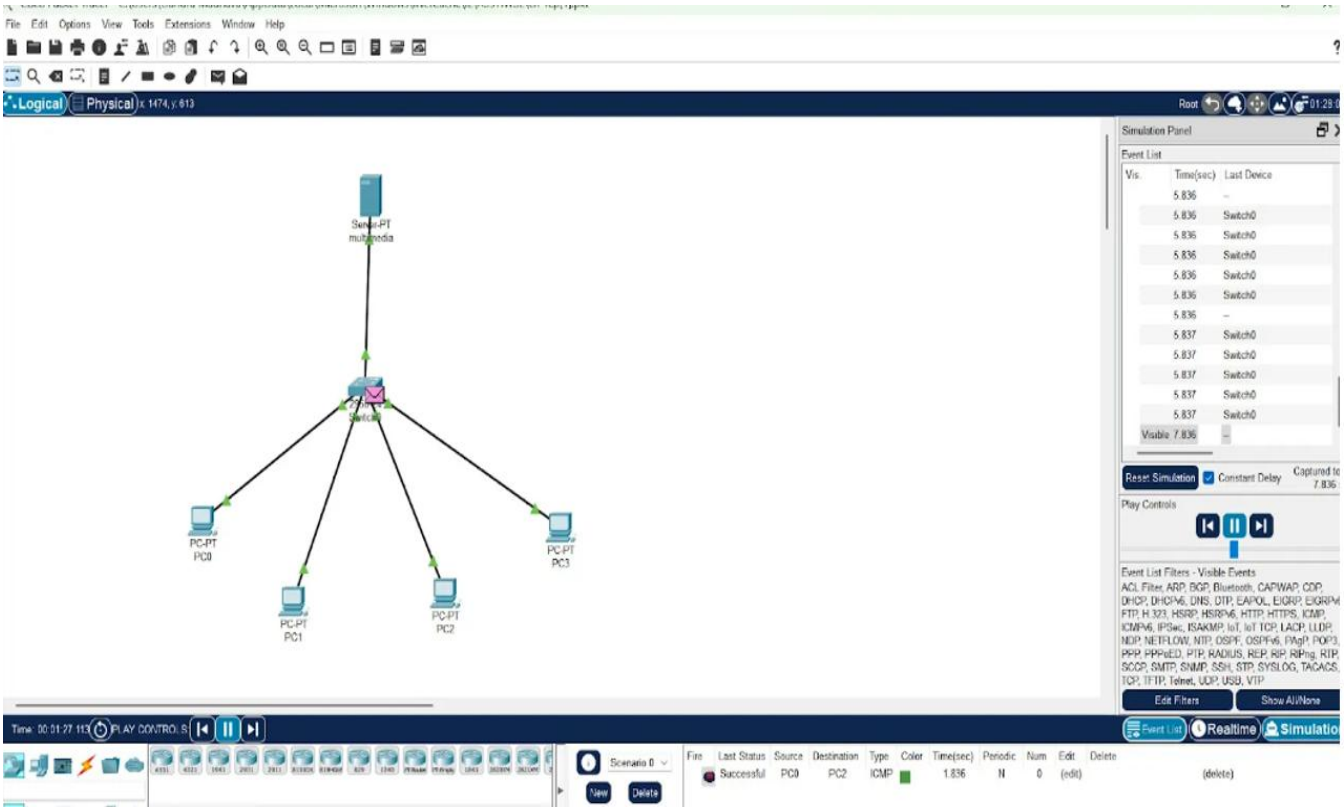
exit

Step 4: Test the connection

Now that the network is set up and configured, we can test the connection between the two computers. Open a command prompt on PC1 and ping PC2 by typing ping 192.168.1.2 in the command prompt. If the ping is successful, it means that the two computers are communicating with each other.

Step 5: Explore TCP functionalities

Now, let's explore the functionalities of TCP. We will use the Netcat utility to establish a TCP connection between the two computers. Netcat is a versatile networking tool that can be used for various purposes, including establishing TCP connections.



Result: Thus the Functionalities and Exploration of TCP using Packet Tracer is designed successfully.

Date:

EXPERIMENT-13

DESIGN THE NETWORK MODEL FOR SUBNETTING – CLASS C ADDRESSING USING PACKET TRACER

AIM: To design the network model for subnetting-class C addressing using packet tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Algorithm:

1. Determine the network requirements: Identify the number of subnets and hosts required for each subnet.
2. Choose a subnet mask: Select a subnet mask that can accommodate the required number of subnets and hosts.
3. Calculate the subnet mask and prefix length: Use the formula 2^p - 2 >= n, where p is the number of host bits and n is the required number of hosts per subnet, to calculate the number of host bits required. Add these host bits to the Class C network address to create the subnet address. The remaining bits in the subnet mask will be the prefix length.
4. Configure the router: Configure the router interface with the subnet address and subnet mask.
5. Configure the hosts: Configure each host with an IP address and subnet mask that matches the subnet address and subnet mask used on the router interface.
6. Test the network: Verify that the hosts can communicate with each other and with devices on other subnets.
7. Monitor network traffic: Use Packet Tracer's built-in network monitoring tools to monitor network traffic and identify any potential issue.

Procedure:

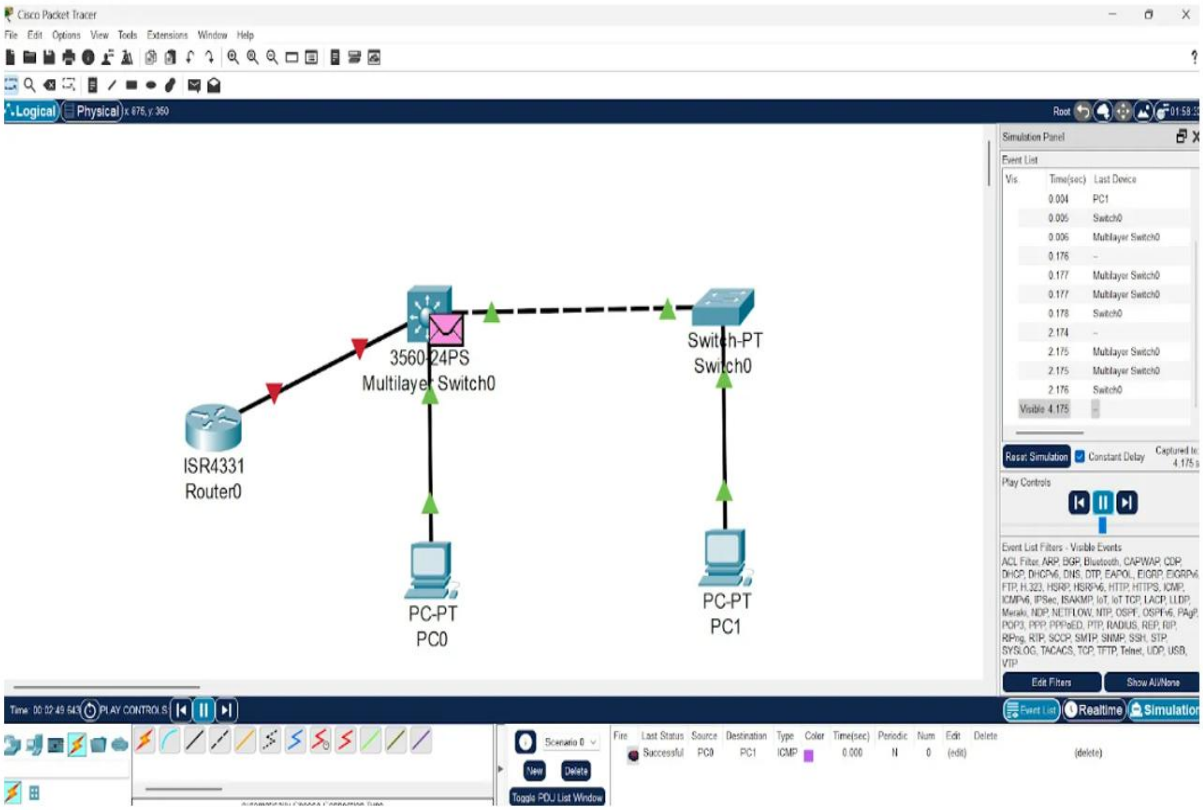
STEP 1: Click on end devices, select generic Pc's drag and drop it on the window. Click on SWITCH drag and drop it on the window.

STEP 2: Select the straight through cable and connect all end device to switch. Assign the IP address for all end devices. (Double click the end device Select → desktop → IP configuration static

STEP 3: Now set the IP address to Host A (192.168.1.1) in static mode. Similarly set IP address for Host B (192.168.1.2) and Host C (192.168.1.3)

STEP 4: To view the IP address, give ipconfig command in command prompt. Using ping command, we can establish communication between two host devices.

STEP 6: Now display the packet transmission in simulation mode.



Result:

There for designing for network model subnetting has been successfully implemented using packet tracer.

Date:

EXPERIMENT: 14 SIMULATING X, Y, Z COMPANY NETWORK DESIGN AND SIMULATE USING PACKET TRACER

Aim: To simulate X,Y,Z company network design and stimulate using packet tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Algorithm:

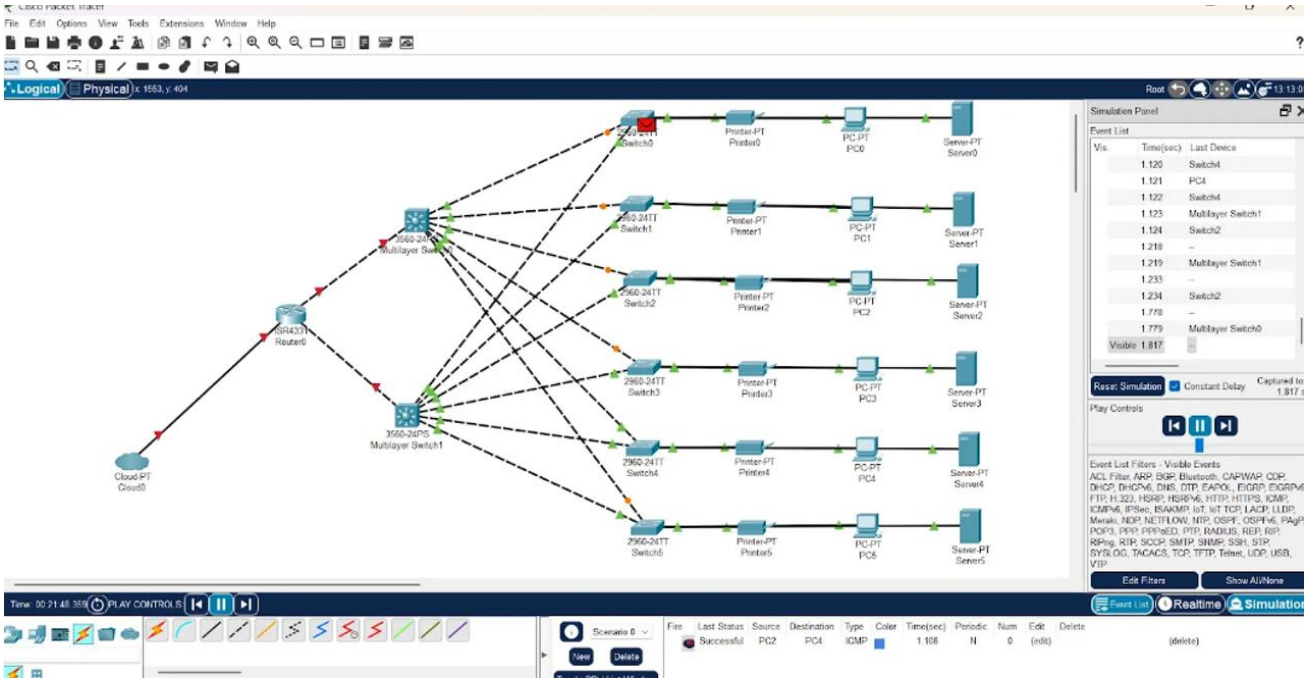
1. Identify the network requirements: Determine the number of users, devices, and servers that will be connected to the network.
2. Create a network diagram: Use a network diagramming tool to create a visual representation of the network design, including the devices, servers, switches, routers, and connections.
3. Configure the routers: Configure the routers with IP addresses, subnet masks, and routing protocols as needed.
4. Configure the switches: Configure the switches with VLANs, and assign ports to each VLAN.
5. Configure the servers: Configure the servers with IP addresses, subnet masks, and any necessary applications or services.
6. Configure the workstations: Configure the workstations with IP addresses, subnet masks, and any necessary applications or services.
7. Configure security: Configure security measures such as firewalls, access control lists, and intrusion detection systems as needed.
8. Test the network: Test the network connectivity by pinging devices and verifying that data can be transmitted between them.
9. Monitor network traffic: Use Packet Tracer's built-in network monitoring tools to monitor network traffic and identify any potential issues.
10. Make adjustments as needed: Make adjustments to the network configuration as needed to improve performance, security, or functionality.

Procedure:

1. Start Packet Tracer: Launch Packet Tracer on your computer.
2. Create a new project: Click on "File" and select "New", then select "Network" from the options.

   3. Add devices: Click on the "Devices" tab in the bottom-left corner of the window, and drag and drop devices onto the workspace. Add devices such as routers, switches, servers, and workstations. 4. Connect devices: Use the "Cable" tool to connect the devices together. Configure the connections as needed.

5. Configure devices: Double-click on each device to open its configuration menu, and configure its settings such as IP address, subnet mask, and routing protocols. Configure security measures such as firewalls, access control lists, and intrusion detection systems as needed.
6. Add applications: Click on the "Applications" tab in the bottom-left corner of the window, and drag and drop applications onto the workstations and servers. Configure the applications as needed.
7. Test the network: Use Packet Tracer's built-in testing tools to verify that the network is working correctly. Test the network connectivity by pinging devices and verifying that data can be transmitted between them.
8. Monitor network traffic: Use Packet Tracer's built-in network monitoring tools to monitor network traffic and identify any potential issues.
9. Make adjustments as needed: Make adjustments to the network configuration as needed to improve performance, security, or functionality.
10. Save the project: Click on "File" and select "Save" to save the project.



Result: Therefore stimulating of companies network designing has been successfully done using packet tracer.

Date:

EXPERIMENT: 15 CONFIGURATION OF DHCP (DYNAMIC HOST CONFIGURATION PROTOCOL) IN PACKET TRACER

Aim: To configure DHCP (dynamic host configuration protocol) in packet tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Algorithm:

1. Start:

• Set up the network topology in Packet Tracer with a DHCP server and DHCP clients connected to a switch.

2. Configure the DHCP server:

- Assign an IP address to the server interface.
- Enable the DHCP service on the server.
- Define the IP address pool range that the server can assign to clients.
- Specify additional DHCP options like default gateway, DNS server, and subnet mask.

3. Configure the switch.

- Enable the switch interfaces that connect to the DHCP clients.

4. Configure the DHCP clients.

- Configure the clients to obtain their IP addresses automatically using DHCP.
- Verify that the clients are set to use DHCP as the preferred method for IP assignment.

5. Client request and server response:

- When a DHCP client boots up or its lease expires, it sends a DHCP discover message as a broadcast on the local network.
- The DHCP server receives the discover message and responds with a DHCP offer message containing an available IP address from the configured IP address pool.
- The server includes other network configuration parameters in the offer message.

6. Client selection and request:

- The client receives multiple offer messages from different DHCP servers if available.
- The client selects one offer and sends a DHCP request message to the chosen server, requesting the offered IP address and confirming other network parameters.

7. Server acknowledgement:

• The DHCP server receives the request message and sends a DHCP acknowledge (ACK) message to the client, confirming the IP address assignment and providing additional network configuration details.

8. Client configuration:

• The client receives the ACK message and configures its network interface with the assigned IP address, subnet mask, default gateway, DNS server, and any other parameters provided by the DHCP server.

9. Lease renewal and expiration:

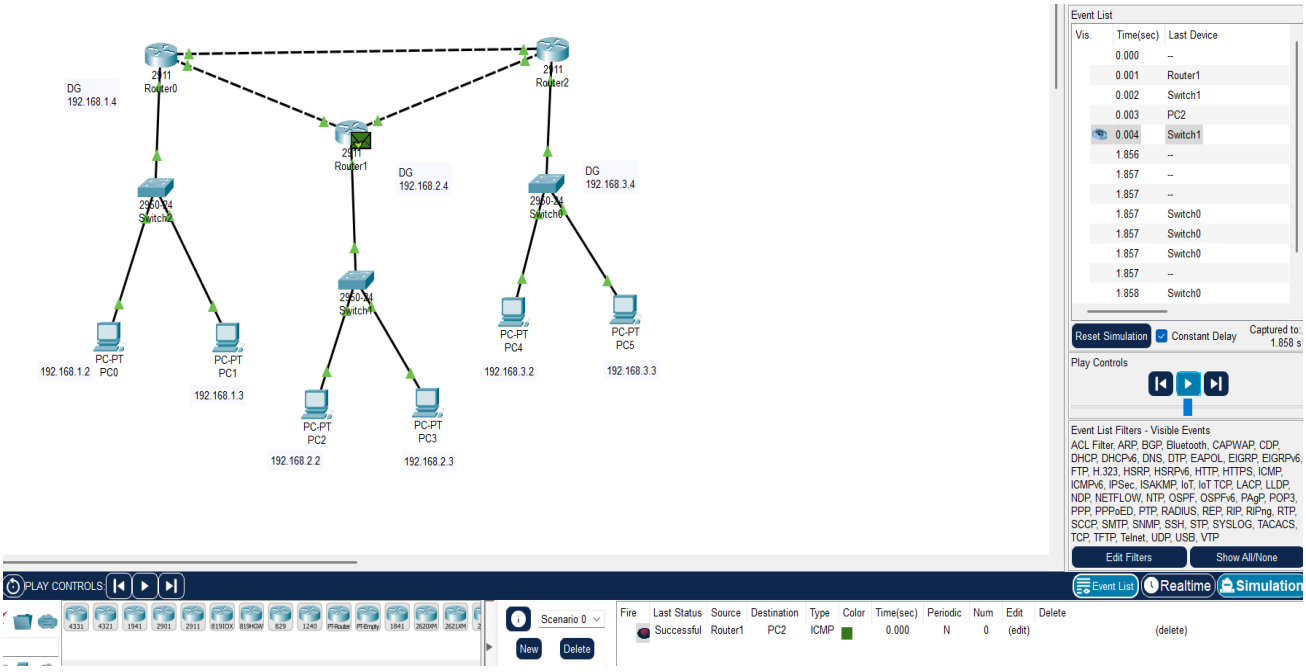
- The client periodically contacts the DHCP server to renew its lease before it expires.
- If the client doesn't renew the lease or is unable to contact the DHCP server, the IP address lease eventually expires, and the IP address returns to the pool for future assignment.

10. End:

Procedure:

1. Launch Cisco Packet Tracer and create a new network topology or open an existing one.
2. Add the necessary network devices to your topology, including a DHCP server, switch, and DHCP clients. Connect them using appropriate cables.
3. Configure the DHCP server:
   - Select the DHCP server device and open its configuration panel.
   - Assign an IP address to the server interface connected to the switch.
   - Enable the DHCP service on the server by checking the "DHCP" option.
   - Define the IP address pool range that the server can assign to clients. Specify the starting and ending IP addresses.
   - Optionally, set other DHCP options like default gateway, DNS server, and subnet mask.
   - Save the configuration.
4. Configure the switch:
   - Select the switch device and open its configuration panel.
   - Enable the interfaces that connect to the DHCP clients. This allows the clients to communicate with the DHCP server.
   - Save the configuration.
5. Configure the DHCP clients:
   - Select each DHCP client device and open its configuration panel.
   - Set the IP address assignment method to "DHCP" or "Obtain an IP address automatically."
   - Save the configuration for each client.
6. Start the simulation:
   - Click the "Start/Stop Simulation" button to start the simulation.
7. Verify DHCP operation:

- Wait for the DHCP clients to boot up or refresh their IP configurations.
- Check if the DHCP clients receive IP addresses from the DHCP server.
- Verify that the clients have the correct IP address, subnet mask, default gateway, and DNS server settings.



Result: Therefore the configuration for DHCP has been successfully executed using packet tracer.

Date:

EXPERIMENT-16 CONFIGURATION OF FIREWALL IN PACKET TRACER

Aim: To configure firewall in packet tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Step 1: Set up the network topology

To begin, we will create a simple network topology consisting of three computers, a router, and a firewall. Open Packet Tracer and drag three PCs, a router, and a firewall onto the workspace. Connect the three PCs to the router using Ethernet cables, and connect the firewall to the router using another Ethernet cable.

Step 2: Configure IP addresses

Next, we will configure IP addresses for the computers. Double-click on each PC to open the configuration window and navigate to the Desktop tab. Click on the IP Configuration icon and enter the IP address and subnet mask for each computer. For example, PC1 can have an IP address of 192.168.1.1 with a subnet mask of 255.255.255.0, PC2 can have an IP address of 192.168.1.2 with the same subnet mask, and PC3 can have an IP address of 192.168.1.3 with the same subnet mask

Step 3: Configure the router

Now, we will configure the router. Double-click on the router to open the configuration window and navigate to the CLI tab. Enter the following commands:

Commands :

enable configure

terminal interface

FastEthernet0/0

ip address 192.168.1.254 255.255.255.0 no

shutdown

exit

Step 4: Configure the firewall

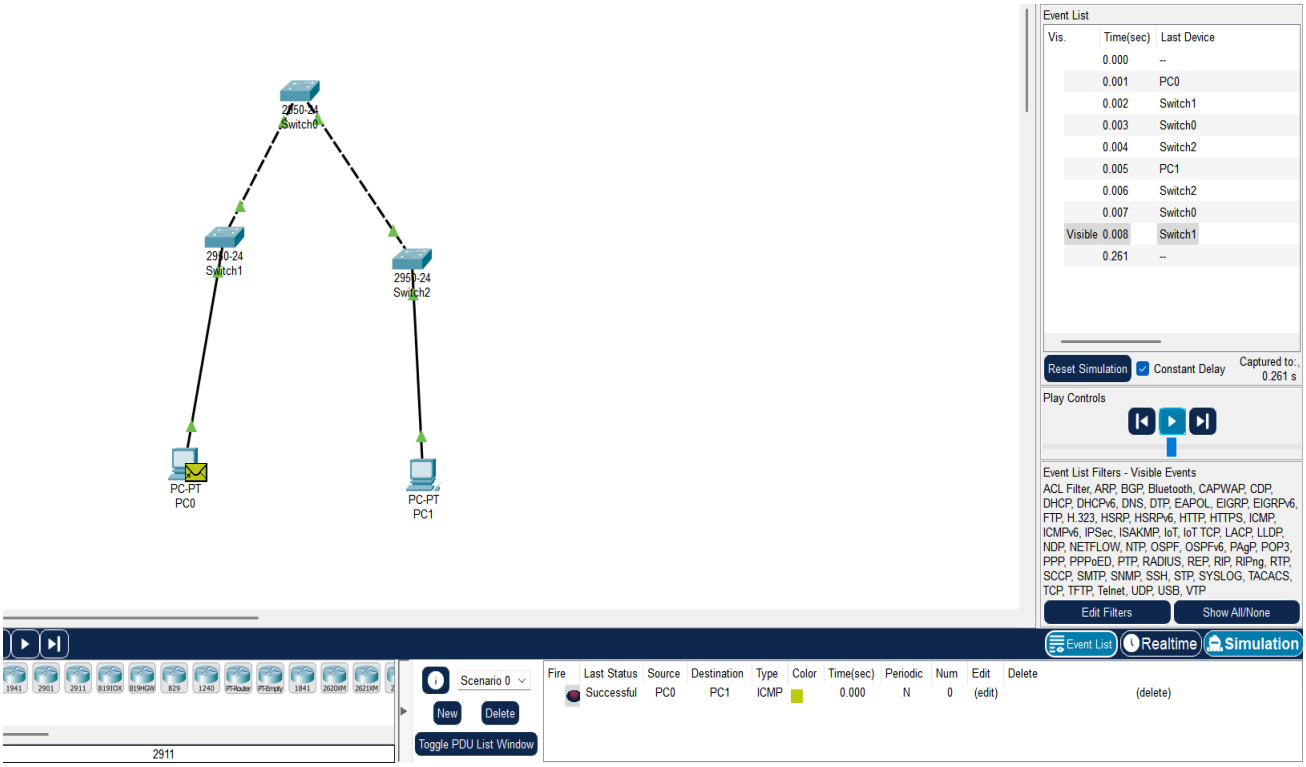
Now, we will configure the firewall. Double-click on the firewall to open the configuration window

Step 5: Test the connection

Now that the firewall is configured, we can test the connection between the computers. Open a command prompt on PC1 and ping PC2 and PC3 by typing ping 192.168.1.2 and ping 192.168.1.3 in the command prompt. If the pings are successful, it means that the computers are communicating with each other.

Step 6: Test the firewall

To test the firewall, try to connect to PC1 from the internet using a protocol or port that is not allowed by the access rule. For example, you can try to connect to PC1 using Telnet on port 23.



Result: Hence the configuration of firewall in packet tracer is successful.

Date:

EXPERIMENT-17 MAKE A COMPUTER LAB TO TRANSFER A MESSAGE FROM ONE NODE TO ANOTHER TO DESIGN AND SIMULATE USING CISCO PACKET TRACER

Aim: To make a Computer Lab to transfer a message from one node to another to design and simulate using Cisco Packet Tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Step 1: Create the network topology

First, we need to create the network topology for the computer lab. In Packet Tracer, drag two computers, a switch, and two routers onto the workspace. Connect the computers to the switch using Ethernet cables, and connect the switch to the two routers using Ethernet cables. The network should look like this:

CODE:

```
PC1        PC2
|          |
Switch -----Router1 ----- Router2
```

Step 2: Configure IP addresses

Next, we will configure IP addresses for the computers. Double-click on each PC to open the configuration window and navigate to the Desktop tab. Click on the IP Configuration icon and enter the IP address and subnet mask for each computer. For example, PC1 can have an IP address of 192.168.1.1 with a subnet mask of 255.255.255.0, and PC2 can have an IP address of 192.168.1.2 with the same subnet mask.

Step 3: Configure the routers

Now, we will configure the routers. Double-click on Router1 to open the configuration window and navigate to the CLI tab. Enter the following commands:

COMMANDS:

enable

configure terminal interface

FastEthernet0/0

ip address 192.168.1.254 255.255.255.0

no shutdown interface Serial0/0/0

ip address 10.0.0.1 255.255.255.252 no

shutdown

exit

Now, double-click on Router2 to open the configuration window and navigate to the CLI tab.

Enter the following commands enable configure terminal interface Serial0/0/0

ip address 10.0.0.2 255.255.255.252

no shutdown interface

FastEthernet0/0

ip address 192.168.2.254 255.255.255.0 no

shutdown

exit

Step 4: Configure routing

We need to configure routing between the routers so that they can communicate with each other.

Enter the following commands on Router1:

enable configure

terminal

ip route 192.168.2.0 255.255.255.0 10.0.0.2 exit

These commands will configure a static route on Router1 to reach the 192.168.2.0/24 network, which is connected to Router2's Fast Ethernet interface.

enable configure

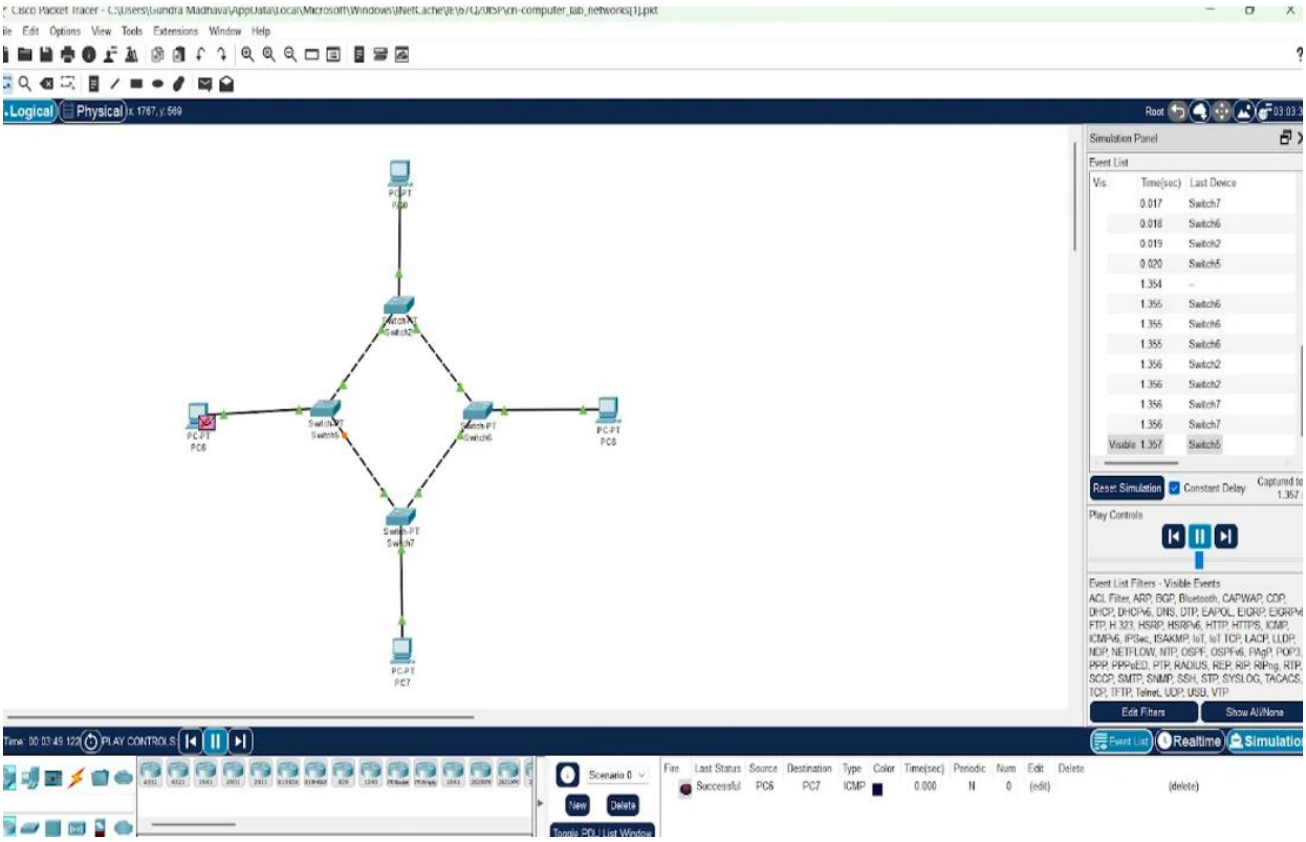
terminal

ip route 192.168.1.0 255.255.255.0 10.0.0.1 exit

Step 5: Send a message

To send a message from PC1 to PC2, open the command prompt on PC1 and type:

ping 192.168.1.2



Result: Hence the message is transferred from one node to another to design and simulate using Cisco Packet Tracer successfully .

Date:

Date:

EXPERIMENT-18

SIMULATE A MULTIMEDIA NETWORK IN CISCO PACKET TRACER

Aim: To simulate a Multimedia Network in Cisco Packet Tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Algorithm:

Procedure:

Step 1: Launch Cisco Packet Tracer and create a new project.

Step 2: Select the appropriate network devices for your multimedia network. You will need computers, switches, routers, and multimedia devices such as IP phones and IP cameras. You can find these devices in the "End Devices," "Switches," "Routers," "Phones," and "IP Cameras" sections of the device list.

Step 3: Design the network topology. Determine the layout of your network and the connections between devices. For example, you can connect the computers, IP phones, and IP cameras to a switch, and then connect the switch to a router for internet connectivity.

Step 4: Drag and drop the devices onto the workspace area. Connect the devices using appropriate cables or wireless connections. For example, use Ethernet cables to connect computers and IP phones to the switch.

Step 5: Configure IP addresses on the devices. Assign IP addresses, subnet masks, and default gateways to the computers, IP phones, and IP cameras. Configure the router's interface with an IP address provided by your ISP or use a DHCP server if available.

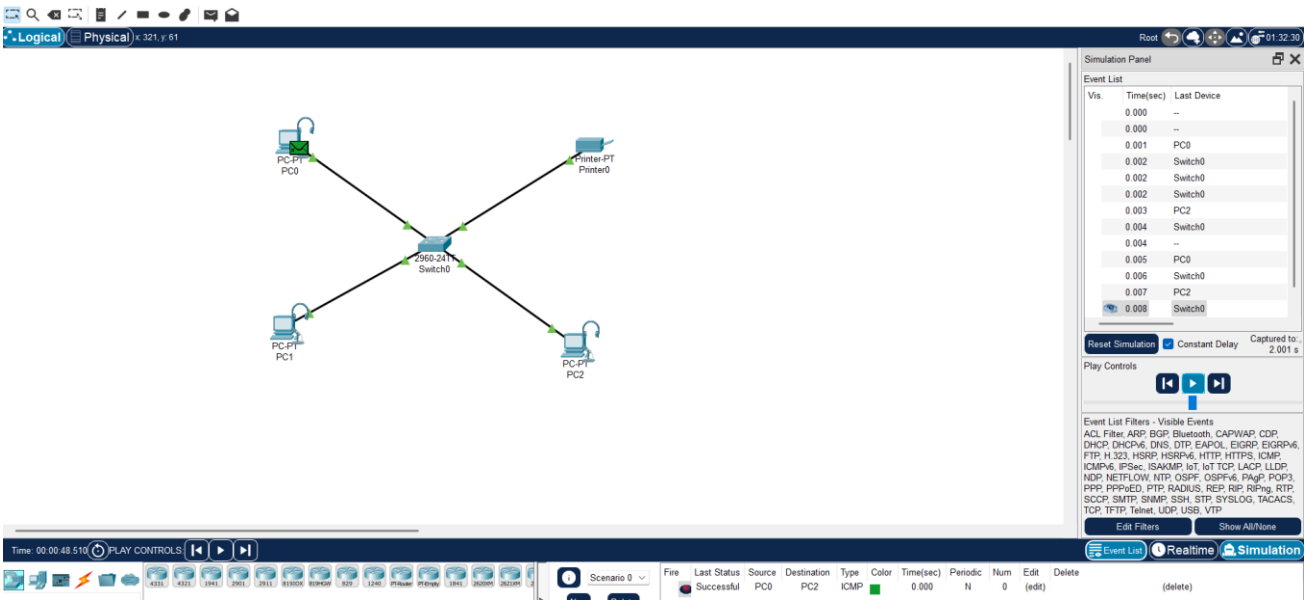
Step 6: Set up multimedia services. Configure the necessary services for multimedia communication, such as VoIP (Voice over IP) for IP phones and streaming protocols for IP cameras. This may involve configuring protocols like SIP (Session Initiation Protocol) for IP phones or RTSP (Real-Time Streaming Protocol) for IP cameras.

Step 7: Test connectivity and multimedia services. Verify that devices can communicate with each other and multimedia services are functioning correctly. For example, try making a call between IP phones or access the video feed from IP cameras.

Step 8: Monitor and troubleshoot. Use the network monitoring tools in Cisco Packet Tracer to observe network traffic and performance. Troubleshoot any issues that arise, such as connectivity problems or audio/video quality degradation.

Step 9: Document the lab experiment. Record observations, configurations, and any issues encountered during the simulation. This documentation will help to analyze the results and make improvements if necessary.

Remember to save your project regularly to preserve your progress. Cisco Packet Tracer provides a simulated environment to experiment with multimedia networks, allowing you to understand the challenges and requirements of such networks in a virtual setting.



Result: Thus a Multimedia Network in Cisco Packet Tracer is simulated successfully.

Date:

EXPERIMENT-19

IOT BASED SMART HOME APPLICATIONS

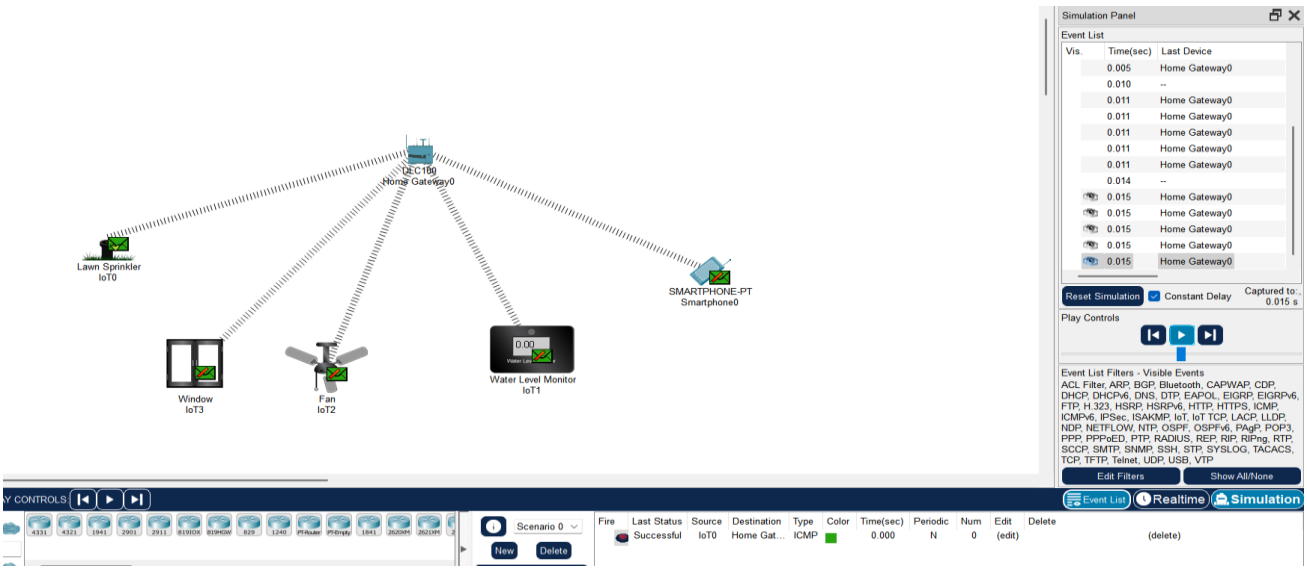
Aim: To implement IoT based smart home applications in Cisco Packet Tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Steps:

1. Create a network topology in Cisco Packet Tracer that includes IoT devices such as sensors, actuators, and gateways.
2. Configure the IoT devices with appropriate IP addresses, subnet masks, and gateway addresses.
3. Set up a communication protocol between the IoT devices using MQTT, CoAP, or any other protocol of your choice.
4. Write a code to collect data from the sensors and send it to the gateway.
5. Use the gateway to process the data and send commands to the actuators.
6. Finally, use a web interface or mobile application to monitor and control the IoT devices. By following these steps an IoT-based smart application in Cisco Packet Tracer , can be created. This can be used for various applications such as home automation, smart cities, and industrial automation.



Result: Thus IoT based smart home applications in Cisco Packet Tracer is implemented successfully.

Date:

EXPERIMENT: 20

IMPLEMENTATION OF IOT BASED SMART GARDENING

Aim: To implement IOT based smart gardening using Cisco packet tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, Connectors.

Procedure:

Step 1: Create a new project in Cisco Packet Tracer and drag a generic IoT device from the IoT devices section onto the workspace.

Step 2: Right-click on the IoT device and select Config/Attributes.

Step 3: In the Configuration tab, select the device's IoT server from the drop-down list. You can choose Cisco IoT Cloud or another cloud service of your choice.

Step 4: In the Attributes tab, add the following attributes:

- Temperature
- Humidity
- Soil Moisture
- Light Intensity

Step 5: Create a soil moisture sensor and a light sensor from the Sensors section of the devices panel. Drag and drop these sensors onto the workspace.

Step 6: Connect the sensors to the IoT device using the wiring tool.

Step 7: Configure the sensors by right-clicking on them and selecting Config/Attributes. Set the sensor type, unit of measurement, and other necessary parameters.

Step 8: Create a water pump and a light bulb from the Actuators section of the devices panel. Drag and drop these actuators onto the workspace.

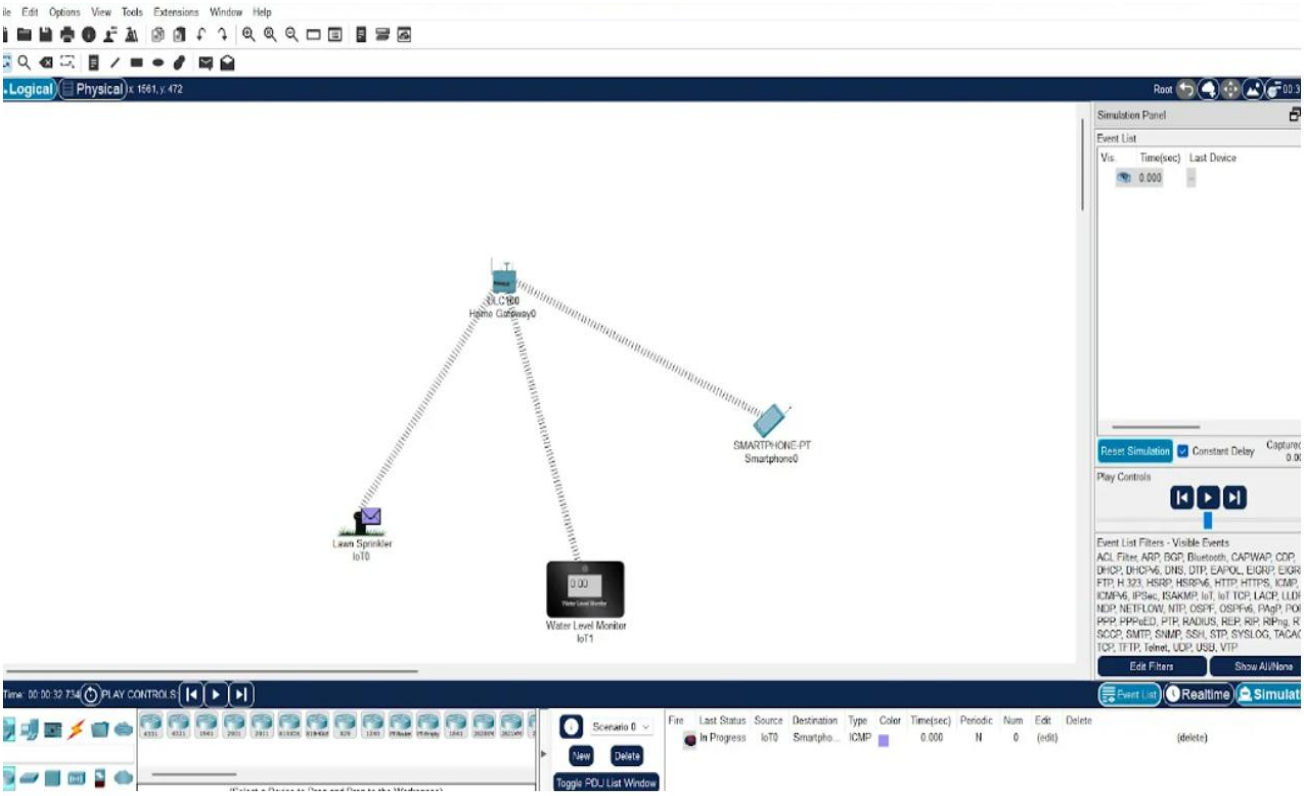
Step 9: Connect the actuators to the IoT device using the wiring tool.

Step 10: Configure the actuators by right-clicking on them and selecting Config/Attributes. Set the actuator type, command, and other necessary parameters.

Step 11: Save the configuration and run the simulation to test your IoT Smart Garden.

Step 12: Monitor the temperature, humidity, soil moisture, and light intensity readings on the IoT device dashboard.

Step 13: Use the dashboard to control the water pump and light bulb based on the sensor readings.



Result: Implementation of smart gardening is carried out using IOT successfully.

Date:

EXPERIMENT: 21

IMPLEMENTATION OF IOT DEVICES IN NETWORKING

Aim: To implement an IOT devices in networking using Cisco Packet Tracer.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

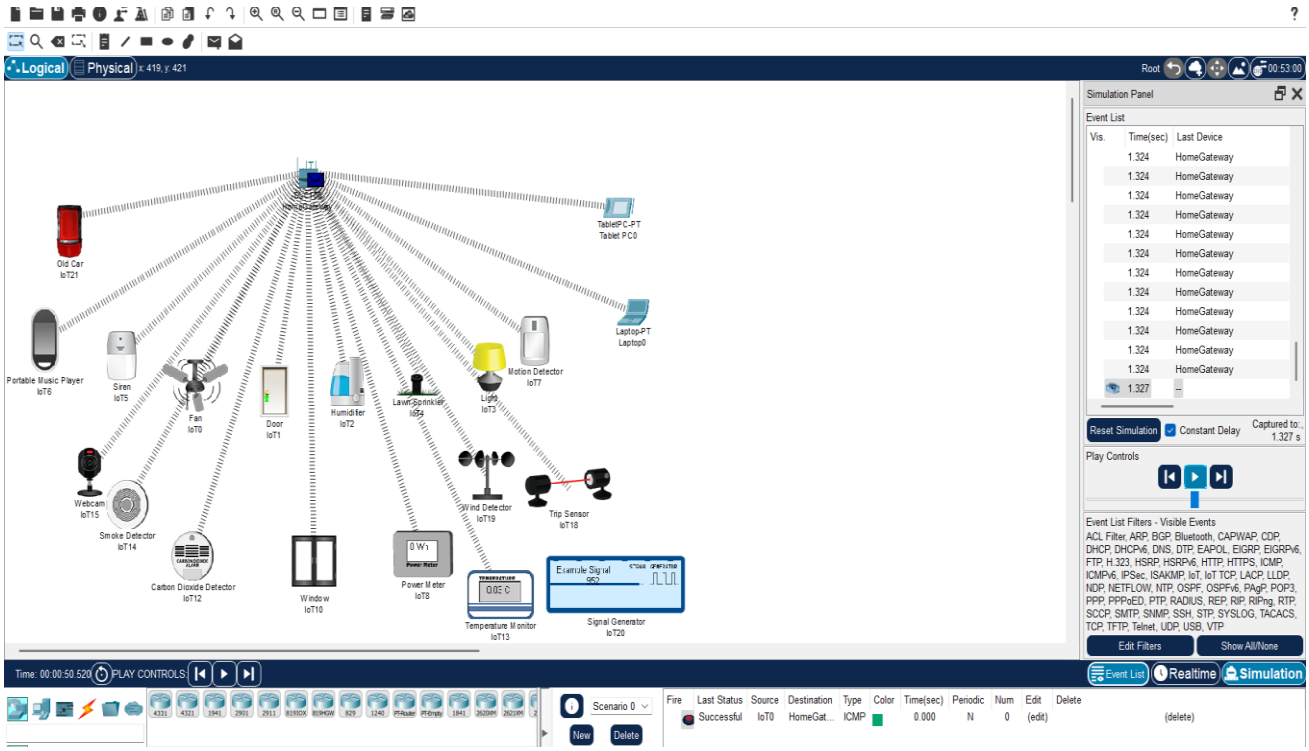
Procedure:

Steps:

1. Open Cisco Packet Tracer and create a new project.

Drag and drop a router from the "Devices" panel onto the workspace area.

2. Connect the router to the Internet by dragging and dropping a "Cloud" device from the "Devices" panel onto the workspace area, and then connecting the router to the cloud using a straight-through cable.
3. Add an IoT device to the network by dragging and dropping a device from the "Devices" panel onto the workspace area. There are various IoT devices available in the "Devices" panel, such as a Raspberry Pi or an Arduino.
4. Connect the IoT device to the router using an Ethernet cable. To do this, click on the IoT device and then click on the "Config" tab. Under the "Interfaces" section, select the Ethernet interface and then click on the "+" button to add a new interface. Connect the new interface to the router.
5. Configure the IoT device by clicking on it and then clicking on the "CLI" tab. This will bring up the command line interface for the IoT device, where you can configure its settings.
6. Test the connectivity of the IoT device by pinging it from the router or from another device on the network.
7. These are just general steps and the specifics of the implementation will depend on the specific IoT device and network configuration you want to create. Additionally, you may need to configure the router and the cloud device to enable Internet connectivity for the IoT device.



Result: Thus an IOT device in networking is implemented using Cisco Packet Tracer successfully.

Date:

EXPERIMENT: 22

IoT based AAA Local and Server based authentication configuration

Aim: Designing an IoT based AAA Local and Server based authentication configuration.

Software/Apparatus required: Packet Tracer/End devices, Hubs, connectors.

Procedure:

Algorithm:

1. Define the Components:

IoT Devices: These are the devices that need to be authenticated and authorized to access the network resources.

Local AAA Server: This server will handle authentication and authorization requests locally. Central AAA Server: This server will provide an additional layer of authentication and authorization for higher-level access control.

2. Setup Local AAA Server:

Configure the local AAA server with the necessary software and databases to handle authentication and authorization requests.

Define user profiles or roles and their associated permissions on the local AAA server.

Set up a secure communication channel between the IoT devices and the local AAA server.

3. Implement Local Authentication:

When an IoT device wants to connect to the network, it sends an authentication request to the local AAA server.

The local AAA server verifies the credentials provided by the IoT device against its user database. If the credentials are valid, the local AAA server generates an authentication token or session key and sends it back to the IoT device.

4. Implement Local Authorization:

Once authenticated, the IoT device sends an authorization request to the local AAA server.

The local AAA server checks the user profile or role associated with the IoT device and verifies if it has the necessary permissions to access the requested resources.

If authorized, the local AAA server sends an authorization response to the IoT device.

5. Configure Central AAA Server:

Set up a central AAA server that will provide an additional layer of authentication and authorization for critical resources or higher-level access control.

Configure the central AAA server with user profiles or roles and associated permissions.

6. Implement Server Authentication:

After local authentication, the IoT device establishes a secure connection with the central AAA server.

The IoT device sends its authentication token or session key to the central AAA server for verification.

The central AAA server validates the authentication token or session key received from the IoT device.

7. Implement Server Authorization:

Once the central AAA server verifies the authentication token or session key, it performs additional authorization checks.

The central AAA server ensures that the IoT device has the necessary permissions to access critical resources or perform higher-level operations.

If authorized, the central AAA server sends an authorization response to the IoT device.

8. Logging and Accounting:

Both the local and central AAA servers maintain logs of authentication and authorization events for auditing and accounting purposes.

They record information such as user/device identification, timestamps, and actions taken.

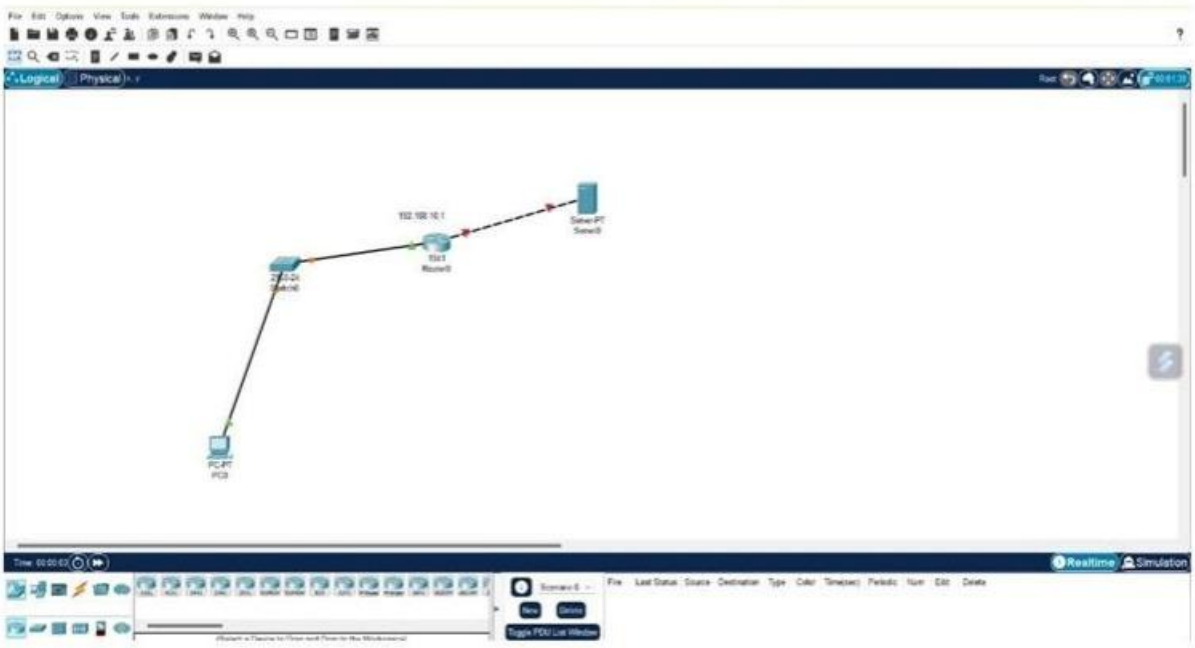
9. Revocation and Updates:

Implement mechanisms to handle credential revocation, such as disabling user accounts or tokens when necessary.

Regularly update user profiles, roles, and permissions in both the local and central AAA servers to reflect changes in the network environment.

Remember to implement appropriate security measures such as encryption, secure communication protocols, and strong password policies to ensure the integrity and confidentiality of the authentication process.

OUTPUT: IoT based AAA Local and Server based authentication configuration.



Result: IoT based AAA Local and Server based authentication is designed successfully.

Date:

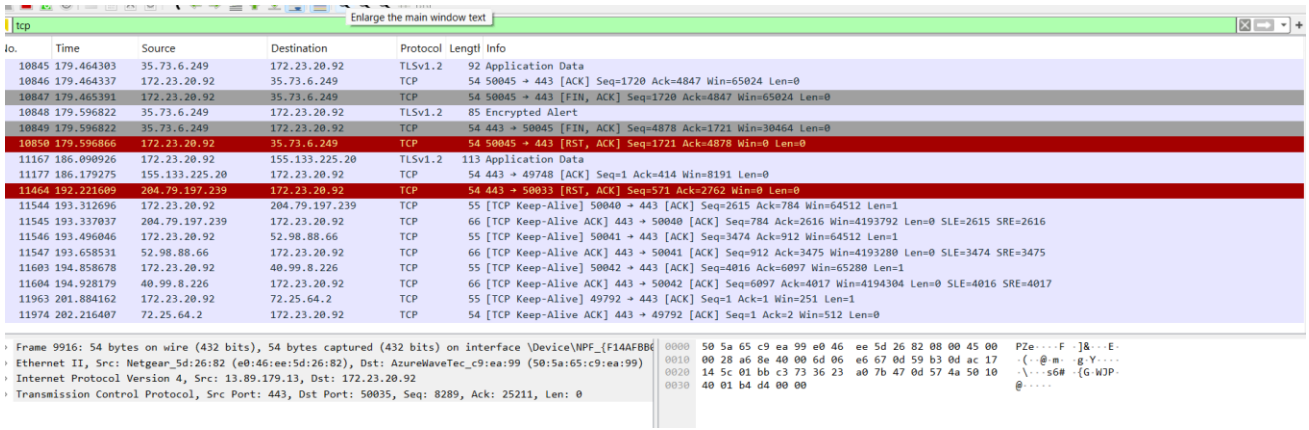
EXPERIMENT: 23 TRANSPORT LAYER PROTOCOL HEADER ANALYSIS USING WIRE SHARK-TCP AND UDP

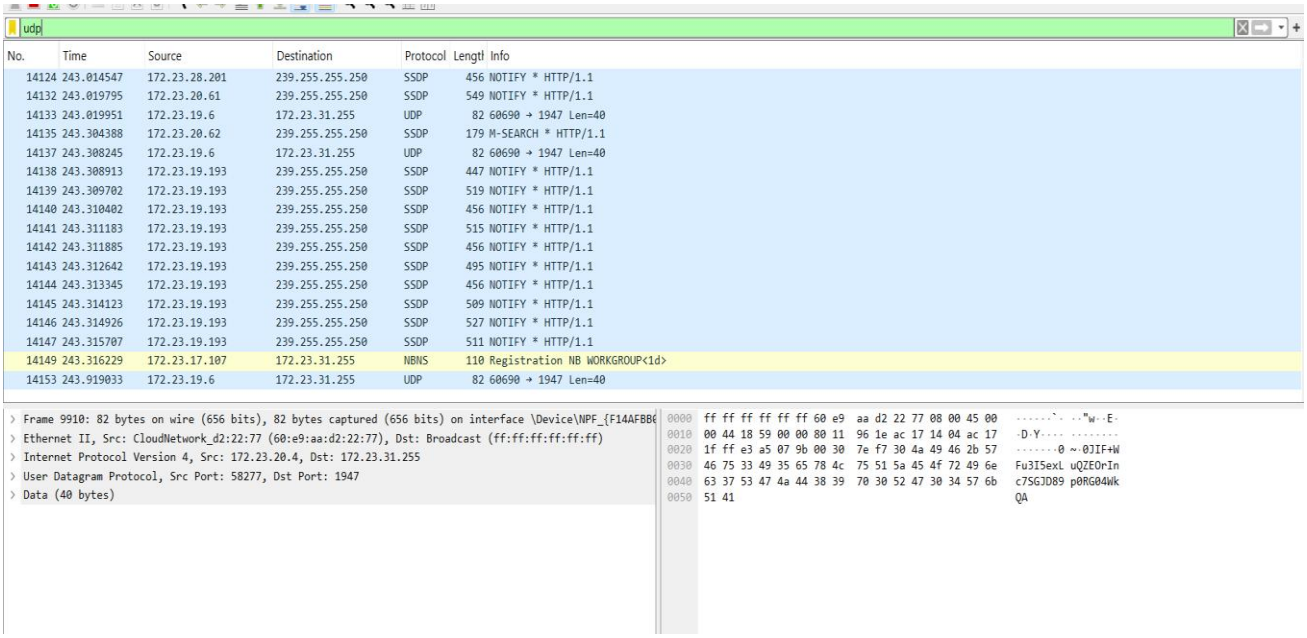
Aim: To analyze capturing of Transport layer protocol header analysis using Wire shark- TCP and UDP.

SOFTWARE USED:

Wire shark network analyzer **Procedure:**

1. Open wire shark.
2. Click on list the available capture interface.
3. Choose the LAN interface.
4. Click on start button.
5. Active packets will be displayed.
6. Capture the packets & select any IP address from the source.
7. Click on the expression and select IPV4 ❼IP addr source address in the field name.
8. Select the double equals (==) from the selection and enter the selected IP source address.
9. Click on apply button.
10. All the packets will be filtered using source address.





Result: Hence, the capturing of packets using wire shark network analyzer was analyzed for TCP and UDP.

Date:

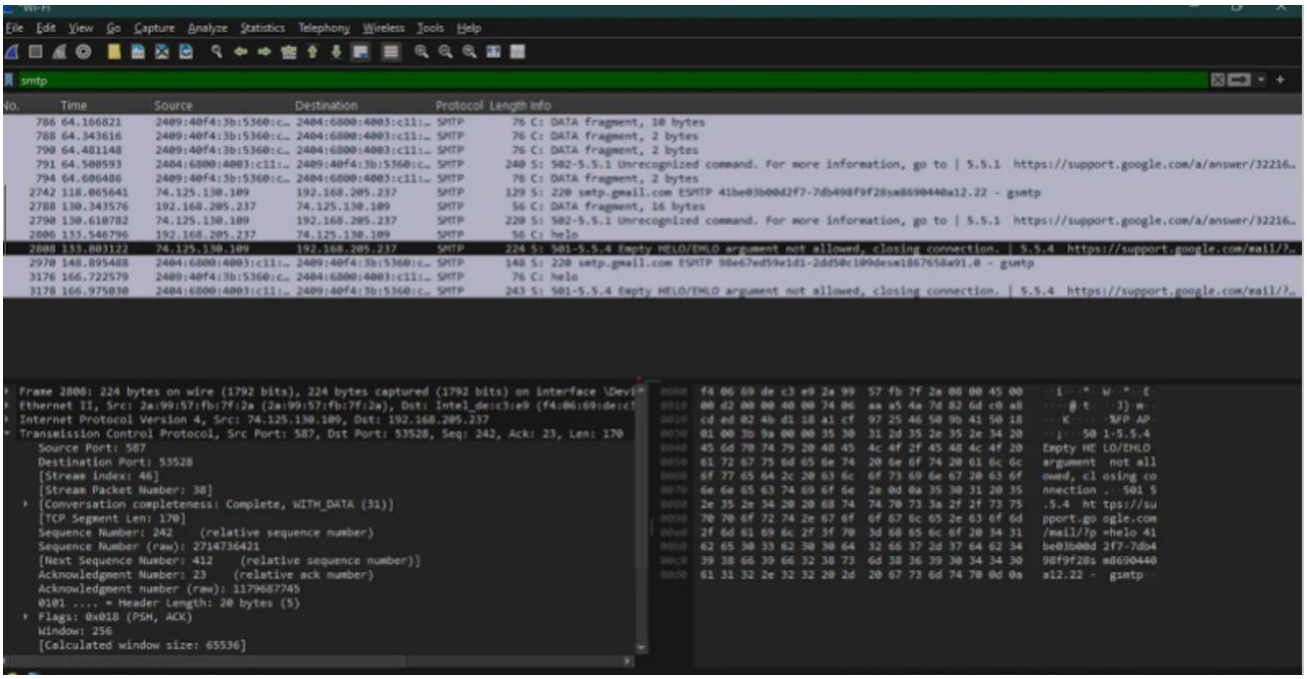
EXPERIMENT-24 NETWORK LAYER PROTOCOL HEADER ANALYSIS USING WIRE SHARK – SMTPAND ICMP

Aim: To analyze capturing of Transport layer protocol header analysis using Wire shark- SMTP and ICMP.

SOFTWARE USED:

Wire shark network analyzer **Procedure:**

1. Open wire shark.
2. Click on list the available capture interface.
3. Choose the LAN interface.
4. Click on start button.
5. Active packets will be displayed.
6. Capture the packets & select any IP address from the source.
7. Click on the expression and select IPV4 ❼IP addr source address in the field name.
8. Select the double equals (==) from the selection and enter the selected IP source address.
9. Click on apply button.
10. All the packets will be filtered using source address.



Result: Hence, the capturing of packets using wire shark network analyzer was analyzed for SMTP and ICMP.

Date:

EXPERIMENT-25

NETWORK LAYER PROTOCOL HEADER ANALYSIS USING WIRE SHARK – ARP AND HTTP

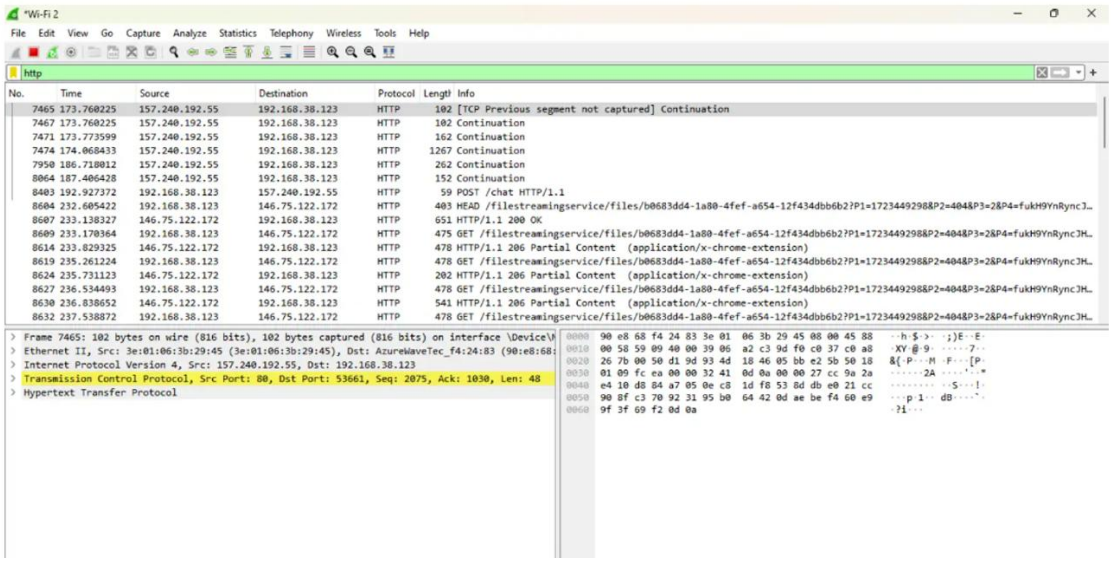
AIM: To analyze capturing of Transport layer protocol header analysis using Wire shark- ARP and HTTP.

SOFTWARE USED:

Wire shark network analyzer

PROCEDURE:

1. Open wire shark.
2. Click on list the available capture interface.
3. Choose the LAN interface.
4. Click on start button.
5. Active packets will be displayed.
6. Capture the packets & select any IP address from the source.
7. Click on the expression and select IPV4 ❼IP addr source address in the field name.
8. Select the double equals (==) from the selection and enter the selected IP source address.
9. Click on apply button.
10. All the packets will be filtered using source address.



Result: Hence, the capturing of packets using wire shark network analyzer was analyzed for ARP and HTTP.

Date:

EXPERIMENT: 26 TO IMPLEMENT DATE AND TIME DISPLAY FROM CLIENT TO SERVER USING TCP SOCKETS IN JAVA/C

Aim: To implement date and time display from client to server using TCP Sockets.

Algorithm:

Server

1. Create a server socket and bind it to port.
2. Listen for new connection and when a connection arrives, accept it.
3. Send server‟s date and time to the client.
4. Read client‟s IP address sent by the client.
5. Display the client details.
6. Repeat steps 2-5 until the server is terminated.
7. Close all streams.
8. Close the server socket.
9. Stop.

Client

1. Create a client socket and connect it to the server‟s port number.
2. Retrieve its own IP address using built-in function.
3. Send its address to the server.
4. Display the date & time sent by the server.
5. Close the input and output streams.
6. Close the client socket.
7. Stop.

Result: Thus date and time is displayed from client to server using TCP Sockets successfully.

Date:

EXPERIMENT: 27 IMPLEMENTATION OF A DNS SERVER AND CLIENT IN JAVA/C USING UDP SOCKET

Aim: To implement a DNS server and client in java using UDP socket **Algorithm:**

Server

1. Create an array of hosts and its IP address in another array
2. Create a datagram socket and bind it to a port
3. Create a datagram packet to receive client request
4. Read the domain name from client to be resolved
5. Lookup the host array for the domain name
6. If found then retrieve corresponding address
7. Create a datagram packet and send ip address to client
8. Repeat steps 3-7 to resolve further requests from clients
9. Close the server socket
10. Stop

Client

1. Create a datagram socket
2. Get domain name from user
3. Create a datagram packet and send domain name to the server
4. Create a datagram packet to receive server message
5. Read server's response
6. If ip address then display it else display "Domain does not exist"
7. Close the client socket **Procedure:**

DNS Server-side implementation:

1. Create a UDP socket using the `socket()` function with the `AF_INET` address family and `SOCK_DGRAM` socket type.
2. Set socket options using the `setsockopt()` function to allow reuse of the address and port.
3. Bind the socket to a specific IP address and port using the `bind()` function.
4. Receive a DNS query from a client using the `recvfrom()` function.

5. Parse the DNS query to extract the requested domain name and record type.
6. Lookup the requested domain name and retrieve the corresponding IP address or other records.
7. Create a DNS response packet with the appropriate format.
8. Send the DNS response to the client using the `sendto()` function with the client address and port obtained from `recvfrom()`.
9. Close the socket using the `close()` function.

DNS Client-side implementation:

1. Create a UDP socket using the `socket()` function with the `AF_INET` address family and `SOCK_DGRAM` socket type.
2. Set the server address and port in a `struct sockaddr_in` structure.
3. Prepare a DNS query packet with the desired domain name and record type.
4. Send the DNS query to the server using the `sendto()` function with the server address and port.
5. Receive the DNS response from the server using the `recvfrom()` function.
6. Parse the DNS response packet to extract the requested information.
7. Process and display the received information as needed.
8. Close the socket using the `close()` function.

DNS Server-side implementation:

1. Create a UDP socket using the `socket()` function with the `AF_INET` address family and `SOCK_DGRAM` socket type.
2. Set socket options using the `setsockopt()` function to allow reuse of the address and port.
3. Bind the socket to a specific IP address and port using the `bind()` function.
4. Receive a DNS query from a client using the `recvfrom()` function.
5. Parse the DNS query to extract the requested domain name and record type.
6. Lookup the requested domain name and retrieve the corresponding IP address or other records.
7. Create a DNS response packet with the appropriate format.
8. Send the DNS response to the client using the `sendto()` function with the client address and port obtained from `recvfrom()`.
9. Close the socket using the `close()` function.

DNS Client-side implementation:

1. Create a UDP socket using the `socket()` function with the `AF_INET` address family and `SOCK_DGRAM` socket type.
2. Set the server address and port in a `struct sockaddr_in` structure.
3. Prepare a DNS query packet with the desired domain name and record type.
4. Send the DNS query to the server using the `sendto()` function with the server address and port.
5. Receive the DNS response from the server using the `recvfrom()` function.
6. Parse the DNS response packet to extract the requested information.
7. Process and display the received information as needed.
8. Close the socket using the `close()` function.

Note: Remember to include the necessary header files (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<netinet/in.h>`, etc.) and handle errors appropriately in your code. Additionally, you may need to implement DNS-specific functions for packet parsing, DNS lookup, and response creation.

Remember to include the necessary header files (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<netinet/in.h>`, etc.) and handle errors appropriately in your code. Additionally, you may need to implement DNS-specific functions for packet parsing, DNS lookup, and response creation.

Result: Thus a DNS server and client in java using UDP socket is implemented successfully.

Date:

EXPERIMENT: 28 DEVELOPING A CLIENT THAT CONTACTS A GIVEN DNS SERVER TO RESOLVE A GIVEN HOSTNAME in JAVA/C

Aim: To develop a client that contacts a given DNS server to resolve a given hostname.

Description: · Get the host name to be resolve using gethostname() · Check the host name using nslookup · Print the IP address, host name, Address length and Address type. · List the addresses stored in lookup

Algorithm :

Step 1. Find the host name by using get host by name()

Step 2. The host name is followed by the list of alias names

Step 3. Pointer points to the array of pointers to the individual address Step

4. For each address call the inet_ntop() and print the returned string

Procedure:

1. Initialize necessary variables and structures:
- Create a socket using the `socket()` function with the `AF_INET` address family and `SOCK_DGRAM` socket type.
- Set the DNS server address and port in a `struct sock addr_in` structure.
- Prepare a DNS query packet with the desired hostname and record type.

2. Send the DNS query to the DNS server:

- Use the `send to()` function to send the DNS query to the DNS server using the socket descriptor and the server address structure.

3. Receive the DNS response from the DNS server:
- Use the `recvfrom()` function to receive the DNS response from the DNS server using the socket descriptor.
- Store the response in a buffer.

4. Parse and process the DNS response:
- Extract the necessary information from the DNS response packet based on the DNS protocol.
- Handle any error conditions, such as a non-existent hostname or failed resolution.

5. Display or use the resolved information:
- Extract the resolved IP address or other relevant data from the DNS response.
- Process and utilize the resolved information as needed in your application.

6. Close the socket:

- Use the `close()` function to close the socket descriptor.

Remember to include the necessary header files (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<netinet/in.h>`, etc.) and handle errors appropriately in your code. Additionally, you may need to implement DNS-specific functions for packet parsing and response handling based on the DNS protocol.

It's important to note that DNS resolution involves dealing with DNS protocol intricacies, including packet format, header fields, and query/response structure. Familiarize with the DNS protocol specifications to ensure correct handling of DNS messages.

Result: Thus a client that contacts a given DNS server to resolve a given hostname is developed successfully.

Date:

EXPERIMENT: 29 CREATING THE APPLICATIONS USING TCP ECHO SERVER AND CLIENT IN JAVA/C

Aim: To create Applications using TCP ECHO SERVER and CLIENT.

Algorithm :

SERVER:

STEP1:Start

STEP2: Declare the variables for the socket

STEP3: Specify the family, protocol, IP address and port number

STEP4: Create a a socket using socket() function

STEP 5: Bind the IP address and Port number

STEP6: Listen and accept the client's request for the connection

STEP7: Read the client's message

STEP8: Display the client's message

STEP 9: Close the socket

STEP10: Stop

CLIENT:

STEP1:Start

STEP2: Declare the variables for the socket

STEP3: Specify the family, protocol IPaddress and port number

STEP4: Create a socket using socket() function

STEP5: Call the connect() function

STEP6: Read the put message

STEP7: Send the input message to the server

STEP 8: Display the server's echo

STEP 9: Close the socket

STEP 10: Stop

Procedure:

TCP Echo Server-side implementation:

1. Create a TCP socket using the `socket()` function with the `AF_INET` address family and `SOCK_STREAM` socket type.
2. Set socket options using the `setsockopt()` function to allow reuse of the address and port.
3. Bind the socket to a specific IP address and port using the `bind()` function.
4. Listen for incoming connections using the `listen()` function.
5. Accept a client connection using the `accept()` function, which returns a new socket descriptor for the accepted connection.
6. Receive data from the client using the `recv()` function on the accepted socket descriptor.
7. Process the received data if necessary.
8. Optionally, send a response back to the client using the `send()` function on the accepted socket descriptor.
9. Close the accepted socket descriptor using the `close()` function.
10. Close the server socket descriptor using the `close()` function.

TCP Echo Client-side implementation:

1. Create a TCP socket using the `socket()` function with the `AF_INET` address family and `SOCK_STREAM` socket type.
2. Set the server address and port in a `struct sockaddr_in` structure.
3. Connect to the server using the `connect()` function with the server socket descriptor and the server address structure.
4. Send data from the client to the server using the `send()` function on the connected socket descriptor.
5. Receive the response from the server using the `recv()` function on the connected socket descriptor.
6. Process and display the received data as needed.
7. Close the connected socket descriptor using the `close()` function.

The echo server simply returns back the received data, allowing the client to see the echoed message. Remember to include the necessary header files (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<netinet/in.h>`, etc.) and handle errors appropriately in the code.

Result: Thus the Applications using TCP ECHO SERVER AND CLIENT is created successfully.

Date:

EXPERIMENT: 30 CREATING THE APPLICATIONS USING TCP CHAT CLIENT AND CHAT SERVER IN JAVA/C

Aim: To create an application using TCP CHAT CLIENT and CHAT SERVER.

Algorithm:

SERVER:

STEP 1: Start

STEP 2: Declare the variables for the socket

STEP 3: Specify the family, protocol, IP address and port number

STEP 4: Create a socket using socket() function

STEP 5: Bind the IP address and Port number

STEP 6: Listen and accept the client's request for the connection

STEP 7: Read the client's message

STEP 8: Display the client's message

STEP 9: Continue the chat

STEP 10: Terminate the chat

STEP 11: Close the socket

STEP 12: Stop **CLIENT:**

STEP 1: Start

STEP 2: Declare the variables for the socket

STEP 3: Specify the family, protocol, IP address and port number

STEP 4: Create a socket using socket() function

STEP 5: Call the connect() function

STEP 6: Read the input message

STEP 7: Send the input message to the server

STEP 8: Display the server's reply

STEP 9: Continue the chat

STEP 10: Terminate the chat

STEP 11: Close the socket

STEP 12: Stop

Result: Thus the application using TCP CHAT CLIENT AND CHAT SERVER is created successfully.

Date :

EXPERIMENT: 31

IMPLEMENTATION OF ARP PROTOCOLS IN JAVA/C

Aim: To implement ARP protocols in JAVA/C.

Algorithm:

STEP1: Start

STEP 2: Declare the variables and structure for the socket

STEP 3: Specify the family, protocol, IP address and port number

STEP 4: Create a socket using socket() function

STEP 5: Call memcpy() and strcpy functions

STEP 6: Display the MAC address

STEP 7: Stop

Implementing the ARP (Address Resolution Protocol) protocol in C involves constructing and sending ARP request and response packets, as well as handling the reception and processing of ARP packets.

Steps involved:

1. Define the necessary structures and constants:
- Define the structure for the ARP header, including fields like hardware type, protocol type, hardware address length, protocol address length, operation, sender hardware address, sender protocol address, target hardware address, and target protocol address.
- Define constants for the ARP hardware type (e.g., Ethernet) and protocol type (e.g., IPv4).

2. Create and send an ARP request packet:
- Create a socket using the `socket()` function with the `AF_PACKET` address family and `SOCK_RAW` socket type.
- Create an ARP request packet using the defined ARP header structure.
- Set the appropriate values for the fields in the ARP header, such as the hardware type, protocol type, operation (request), sender hardware and protocol addresses, and target protocol address. - Create and set a `struct sockaddr_ll` structure to specify the interface index, destination MAC address, and other necessary information for sending the packet.
- Use the `sendto()` function to send the ARP request packet using the socket descriptor and the destination address structure.

3. Receive and process ARP packets:

- Create a socket using the `socket()` function with the `AF_PACKET` address family and `SOCK_RAW` socket type.
- Use a loop to continuously receive packets using the `recvfrom()` function and the socket descriptor.
- Extract the received packet and analyze its contents.
- Parse the ARP header from the received packet and examine the operation field to determine if it is an ARP request or response.
- Process the ARP packet accordingly, such as updating an ARP cache or responding to ARP requests.

Note: Implementing the full functionality of ARP, including caching, table management, and handling ARP requests and responses in a network environment, can be complex. The above steps provide a high-level overview, but a complete implementation would require further details and handling of specific use cases.

Remember to include the necessary header files (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<netinet/if_ether.h>`, etc.) and handle errors appropriately in your code. Additionally, consider working with lower-level networking libraries or using a network packet capture library like libpcap for handling raw sockets and packet capturing.

Result: Thus the ARP protocols in JAVA/C are implemented successfully.

EXPERIMENT: 32

IMPLEMENTATION OF BIT STUFFING MECHANISM USING JAVA/C

Aim: To implement Bit stuffing mechanism using C program.

Bit suffering: It is a technique used in communication system to prevent data loss or corruption during transmission.

It involves inserting one or more extra bits into a data packet to differentiate it from the control characters.

Bit suffering is implemented using bitwise operators in c programming language.

In this code, the `bit Stuffing` function takes an input byte array, its length, an output byte array, and a pointer to the output length variable. It performs bit stuffing on the input data and stores the stuffed data in the output array.

The main logic of the bit stuffing is implemented using bitwise operations. The input data is processed byte by byte, and each bit is checked for consecutive 1's. If five consecutive 1's are found, a 0 bit is stuffed into the output frame. The flag sequence (01111110) is added at the beginning and end of the output frame.

In the `main` function, an example input frame is provided, and the bit stuffing is performed by calling the `bit Stuffing` function. The input and output frames are then printed for verification.

Note that in this example, the input frame is hard-coded, and the output frame is printed in hexadecimal format for better readability. You can modify the input frame and test the code with different inputs.

Result : Therefore bit suffering mechanism has been successfully implemented using c program.

EXPERIMENT: 33

IMPLEMENTING THE APPLICATIONS USING TCP FILE TRANSFER IN JAVA/C

Aim: To implement the applications using TCP file transfer in java.

Algorithm:

1. Start the program.
2. Declare the variables and structures required.
3. A socket is created and the connect function is executed.
4. The file is opened.
5. The data from the file is read and sent to the server.
6. The socket is closed.
7. The program is stopped.

Steps to implement a TCP file transfer:

1. Set up the TCP client-server connection:

- Create a server socket using the `socket()` function.
- Bind the server socket to a specific IP address and port using the `bind()` function.
- Listen for incoming client connections using the `listen()` function.
- Create a client socket using the `socket()` function.
- Connect the client socket to the server using the `connect()` function.

2. Server-side implementation:

- Accept the client connection using the `accept()` function on the server side.
- Open the file to be transferred in binary mode using `fopen()`.
- Read the contents of the file in chunks and send them over the TCP connection using the `send()` function.
- Close the file using `fclose()`.

3. Client-side implementation:

- Receive the file data from the server using the `recv()` function on the client side.
- Write the received data to a file on the client side using `fwrite()`.
- Continue receiving and writing data until the entire file is received.
- Close the file using `fclose()`.

Result: Thus the applications using TCP file transfer in java is completed successfully.

Date:

EXPERIMENT: 34 IMPLEMENTING THE SIMULATION OF ERROR CORRECTION CODE – CRC IN JAVA/C

Aim: To implement the simulation of error correction code – CRC in java.

Steps:

1. Define the CRC polynomial and bit length:
- Choose a CRC polynomial. Common choices are CRC-16 (16 bits) or CRC-32 (32 bits).
- Define the polynomial value as a constant in your code.
2. Implement the CRC calculation function:
- Write a function that takes the input data and calculates the CRC value based on the chosen polynomial.
- The function should iterate through each bit of the input data and perform XOR operations with the polynomial.
- At the end of the calculation, the CRC value should be returned.
3. Test the CRC calculation function:
- Create test data with known CRC values.
- Use the CRC calculation function to calculate the CRC for the test data and compare it with the expected CRC value.
- Verify that the CRC calculation function produces the correct CRC values for the test data. In this code, the `calculate CRC` function takes an input data array and its length. It performs the CRC calculation by iterating through each bit of the data and performing XOR and shift operations. The function returns the calculated CRC value.

In the `main` function, a test data array is defined along with the expected CRC value. The `calculate CRC` function is then called to calculate the CRC for the test data. The calculated and expected CRC values are printed, and a comparison is made to check if they match.

Result: The simulation of error correction code – CRC in java is implementation successfully.

EXPERIMENT: 35
IMPLEMENTION OF SLIDING WINDOW PROTOCOL IN JAVA/C

Aim: To implement sliding window protocol in java.

Steps:

1. Set up the client-server connection:
- Create a server socket using the `socket()` function.
- Bind the server socket to a specific IP address and port using the `bind()` function.
- Listen for incoming client connections using the `listen()` function.
- Create a client socket using the `socket()` function.
- Connect the client socket to the server using the `connect()` function.

2. Implement the sliding window protocol:
- Define the window size (number of packets) and other necessary parameters.
- Split the data to be sent into packets, and assign a sequence number to each packet.
- Implement a sender and receiver window to keep track of sent and acknowledged packets.
- Send packets from the sender window and maintain a timer for each packet.
- Receive packets at the receiver, send acknowledgments (ACK) for correctly received packets, and discard duplicates.
- Update the sender and receiver window based on acknowledgments.
- Repeat the process until all packets have been sent and acknowledged.

Result: Hence implementation of sliding window protocol was done successfully in java.